SoK: FuzzCheck.AI: On the Limitations of DNN Fuzzing to Discover Security Failures

***Abstract*—Deep neural networks (DNNs) are widely deployed in safety- and security-critical applications and remain vulnerable to adversarial manipulation. DNN fuzzing has emerged as a practical approach for systematically discovering model failures. However, existing DNN fuzzing research remains fragmented across goals, designs, and evaluation practices, making it unclear whether current fuzzers satisfy core security testing requirements, whose omission may lead to a misleading sense of security. To address this gap, we present FuzzCheck.AI, a structured evaluation framework with six DNN security testing metrics (failure severity, targeted attack discovery, input plausibility, failure reproducibility, failure diagnostics, and attack transferability) and a codebook-guided, semi-automated assessment workflow combining LLM-based evidence extraction with expert review. Applying FuzzCheck.AI to 38 representative DNN fuzzing papers, we uncover systematic blind spots in current fuzzing designs. Most fuzzers focus on low-severity robustness errors, rarely support targeted or transferable attacks, and provide limited failure diagnostics. A substantial fraction overlook cumulative mutation effects on input plausibility or serialization effects, leading to implausible or non-reproducible failures. No evaluated fuzzer achieves high compliance across all metrics, and 63% (24/38) exhibit low compliance on three or more. Based on these findings, we derive actionable recommendations to guide the design of DNN fuzzers for comprehensive security testing.**

I. Introduction

Deep neural networks (DNNs) are increasingly deployed in safety- and security-critical settings, including autonomous driving, speech interfaces, and large language model–based systems [1], [2], [3]. Despite impressive benchmark performance, these systems remain vulnerable to adversarial manipulations. Small, often imperceptible perturbations, such as pixel-level noise in images, waveform distortions in audio, or subtle token manipulations in text, can induce mispredictions, unsafe actions, or violations of safety and security policies [4], [5], [6], [7], [8], [9]. This gap between benchmark performance and model behavior under adversarial or uncommon yet valid inputs has motivated substantial work on automated DNN testing [10], [11], [12], [13], [14], [15], [16], highlighting the need for systematic approaches that go beyond static benchmark evaluation.

DNN fuzzing has emerged as a practical approach for probing model behavior under diverse, systematically mutated inputs. Inspired by traditional software fuzzing [17], [18], [19], recent DNN testing frameworks adapt fuzzing principles to deep learning models [20], [21], [22], [23]. At a high level, DNN fuzzers iteratively mutate seed inputs and leverage feedback signals from the target model to guide exploration toward previously underexplored or failure-prone behaviors [10], [11], [24]. Existing fuzzers differ substantially in their design choices. They vary in how test inputs are mutated, ranging from metamorphic transformations and gradient-guided perturbations to generative input synthesis [10], [21], [25]. They also differ in how exploration is guided, for example by selecting inputs for subsequent mutation that have previously triggered faults or expanded internal model activation coverage [21], [23]. In addition, fuzzers differ in their oracle designs, which specify failure conditions. Fuzzers also differ in their access assumptions, including whitebox, greybox, and blackbox settings [20], [11], [26]. Together, these design choices shape how effectively a fuzzer can discover DNN security failures such as jailbreaks, data leakage, bypassing model defenses, targeted attacks, and cross-model transferable attacks.

DNN fuzzing approaches often report high failure discovery efficiency, yet provide limited evidence on whether the discovered failures satisfy essential but under-examined security testing requirements. It is often unclear whether fuzzers discover targeted or cross-model transferable attacks, whether discovered failures bypass deployed defense mechanisms, or whether they expose underlying model vulnerabilities, such as reliance on spurious features or dataset bias. Moreover, discovered failures are meaningful only when the corresponding fault-inducing inputs remain plausible under realistic threat models and when they remain reproducible. Fuzzers that overlook any of these requirements may discover numerous failures, giving the impression of thorough testing, while leaving critical aspects of DNN security insufficiently examined and thereby providing a misleading sense of security.

As a rapidly emerging area, DNN fuzzing research remains fragmented, with substantial variation in mutation strategies, exploration mechanisms, and oracle designs. Consequently, it remains unclear how specific design choices shape a fuzzer's ability to satisfy different security testing requirements. Without a systematic analysis of fuzzing approaches, this fragmented landscape risks duplicated effort and blind spots in critical security testing dimensions. A systematization at this stage is therefore essential to clarify the capabilities and limitations of existing DNN fuzzing approaches in discovering security failures and to guide the design of frameworks for comprehensive security testing. To the best of our knowledge, no prior work has systematically examined how different DNN fuzzing approaches satisfy these security testing requirements.

To address this gap, we distill six DNN security testing metrics (failure severity, targeted attack discovery, input plausibility, failure reproducibility, failure diagnostics, and attack transferability) from prior work on machine learning security and develop a qualitative evaluation framework to assess DNN

TABLE I
SUMMARY OF KEY FINDINGS AND RECOMMENDATIONS.

Key Finding	Recommendation
Only 18.8% discover high-severity failures, while 65.6% focus on low-severity robustness errors.	Incorporate feedback from model defense mechanisms and encode security and policy violations in the oracle to discover high-severity failures.
Only 9.4% support targeted attack discovery, while 68.8% focus on untargeted failures.	Use target-aware exploration and directional mutation mechanisms to guide fuzzing toward attacker-chosen outputs.
37.5% overlook cumulative effect of mutation across iterations, leading to implausible failures.	Enforce plausibility relative to the original seed, or use auxiliary models to filter implausible mutations.
Over half of fuzzers ignore serialization effects (e.g., clipping, rounding), resulting in fragile, non-reproducible failures.	Design reproducibility-aware mutation strategies that explicitly account for serialization effects (e.g., clipping, rounding).
Most DNN fuzzing papers (96.8%) provide only partial or no failure diagnostics.	Evaluate fuzzers on models with known vulnerabilities and analyze whether discovered failures correlate with them.
Transferable attack discovery is largely unsupported, with only one fuzzer explicitly designed for it.	Incorporate multi-model feedback to guide mutation and exploration toward transferable failure discovery.
No fuzzer achieves full compliance; 59.4% fail on three or more metrics.	Integrate fuzzing components with complementary strengths to satisfy all core security testing requirements.

fuzzing papers/techniques along these dimensions. In doing so, we make several key contributions, and offer critical findings and recommendations to design better DNN fuzzers in the future, as outlined below:

1) We present FUZZCHECK.AI, a structured evaluation framework that defines six metrics capturing core DNN security testing requirements and provides a codebook with concrete evaluation guidelines for assessing DNN fuzzing papers.
2) We design a semi-automated, two-stage assessment workflow in which LLMs extract metric-relevant evidence and propose preliminary assessments guided by the FUZZCHECK.AI codebook, which experts validate and finalize to ensure reliable evaluation of DNN fuzzing papers.
3) We propose a taxonomy that categorizes DNN fuzzing approaches along multiple design dimensions (oracle design, mutation strategy, exploration strategy, and access level), enabling systematic analysis of how design choices influence security testing metrics.
4) Applying FUZZCHECK.AI to 32 representative DNN fuzzing papers, we identify recurring blind spots and derive actionable research directions, summarized in Table I, to guide the design of fuzzers for comprehensive DNN security testing.

II. BACKGROUND

A. DNN Fuzzing

DNN fuzzing is an automated testing paradigm that systematically probes the behavior of deep neural networks by iteratively generating and evaluating test inputs. Inspired by traditional software fuzzing [17], [18], [19], DNN fuzzers typically operate in a loop that starts from a set of seed inputs, applies mutations to produce new candidate inputs, executes the target model on these inputs, and uses feedback signals to guide subsequent exploration toward underexplored or failure-prone behaviors. Unlike benchmark-based evaluation, which measures performance on a fixed dataset, DNN fuzzing focuses on adaptive exploration of the input space [10], [20], [21].

At a high level, DNN fuzzers consist of three core components: mutation, exploration, and oracle. **Mutation** determines how new test inputs are generated from existing seeds or synthesized from scratch. **Exploration** governs how candidate inputs are prioritized during fuzzing based on feedback signals obtained from model execution. **Oracle** specifies how a fuzzer infers whether a generated input exposes incorrect, unsafe, or security-violating behavior in the DNN model. Depending on the level of access to the target model, DNN fuzzing can operate in white-box, grey-box, or black-box settings, each providing different degrees of visibility into internal model information [27].

DNN fuzzing is closely related to, but distinct from, other DNN testing approaches. Metamorphic testing (MT) applies predefined, semantics-preserving transformations and tests DNN models by checking prediction-level consistency between an input and its transformed variants, but does not perform iterative, feedback-guided exploration [13], [14], [28], [29]. Optimization-based adversarial attacks optimize small, semantics-preserving perturbations to induce targeted or untargeted mispredictions, emphasizing worst-case evasion objectives rather than systematic exploration of the model behavior space [30], [31], [8], [32], [33], [34], [35]. Coverage-based DNN testing measures test adequacy by quantifying how thoroughly a dataset exercises a model's internal behavior using coverage metrics defined over neuron activation states [36], [37], [15], [38].

B. DNN Fuzzing vs. Software Fuzzing

While inspired by traditional software fuzzing, DNN fuzzing differs in several fundamental ways. Unlike traditional software, where execution paths and code coverage provide a clear notion of exploration progress, DNNs operate in continuous, high-dimensional activation spaces in which each input induces a distinct pattern of internal activations [10], [21]. As a result, DNN fuzzers approximate exploration progress using heuristic feedback signals derived from model internals or outputs [10], [15], [39].

Another distinction lies in how fuzzers define an oracle to determine whether a test input exposes incorrect, unsafe, or security-violating behavior. In traditional software fuzzing, program crashes, assertion violations, or memory errors provide an unambiguous indication of such behavior. In contrast, DNN fuzzing lacks reliable ground-truth labels for mutated or synthesized inputs, making it difficult to directly determine whether a model's behavior violates security

or safety properties [21]. Consequently, DNN fuzzers infer failures using alternative oracle definitions, such as violations of safety or security policies [23], [24], inducing attacker-chosen outputs, inconsistencies across semantically equivalent inputs [12], [39], or divergence among models performing the same task [40], [41].

A further distinction arises in mutation strategy. In DNN fuzzing, mutation is designed to discover inputs that can fool the learned model into exhibiting safety- or security-violating behavior, whereas traditional software fuzzing uses mutation primarily to exercise execution paths that lead to program failures. As a result, the semantic plausibility of mutated inputs is largely irrelevant in traditional fuzzing but central to DNN fuzzing [42], [43], [20]. The security relevance of a discovered failure depends on whether it is triggered by inputs that preserve perceptual or semantic plausibility. Implausible or semantically invalid inputs may induce arbitrary model behavior but do not necessarily reflect exploitable weaknesses under realistic threat models, as practical adversarial manipulations are typically constrained to be imperceptible in order to evade detection [30], [21], [44].

III. Related Work

Several comprehensive surveys have explored the testing and evaluation of machine learning (ML) systems, each emphasizing different perspectives on robustness, safety, and security. Riccio et al. [45] and Zhang et al. [27] conducted systematic surveys of ML testing, outlining major techniques and challenges in test input generation, oracle design, adequacy criteria, and evaluation of testing effectiveness. Braiek and Khomh [46] and Huang et al. [47] reviewed how classical software-testing principles, such as coverage metrics, metamorphic testing, and formal verification, have been adapted to assess the reliability and safety of deep models. More recently, Guo et al. [16] surveyed coverage-guided testing for deep learning, systematically categorizing 89 studies by coverage criteria, test input generation, and optimization strategies. They also review evaluation practices and highlight open challenges in correlating coverage with fault detection, improving generalizability, and standardizing benchmarks.

While prior surveys primarily focus on categorizing DNN testing techniques and analyzing design and evaluation challenges, they do not systematically examine how existing DNN fuzzing designs satisfy core security testing requirements. To address this gap, we introduce a structured evaluation framework with six metrics and apply it to 32 representative DNN fuzzing papers, revealing systematic blind spots and design limitations that are not visible through conventional survey perspectives. Based on these findings, we distill actionable recommendations to guide the design of DNN fuzzers for comprehensive security testing.

IV. Our FuzzCheck.AI Framework

We introduce FuzzCheck.AI, a structured evaluation framework for assessing how different DNN fuzzing designs satisfy core security testing requirements. The framework consists of six security testing metrics that capture core requirements in DNN security testing. The framework also includes a codebook that specifies concrete evaluation guidelines for assessing DNN fuzzing papers on these metrics, enabling consistent and transparent evaluation. As illustrated in Figure 1, the codebook is applied to assess each paper against all six metrics based on its described fuzzing design and reported empirical results. These assessments are then aggregated to analyze trade-offs and blind spots across fuzzing designs, examine how different design choices support or constrain security testing requirements, and distill actionable insights to guide the development of future DNN fuzzing approaches.

A. Development of FuzzCheck.AI

We distilled the six FuzzCheck.AI metrics by analyzing evaluation practices and security testing goals in prior work on DNN testing and machine learning security. In particular, we examined how existing studies define DNN model security failures, attack surfaces, and threat models, and how they reason about the causes of discovered failures, to identify core DNN security testing requirements. These requirements include uncovering high-severity security failures [23], [8], supporting targeted attack discovery [8], [9], ensuring the plausibility of fault-inducing inputs [44], [12], maintaining failure reproducibility [48], providing explanations for discovered failures [39], [49], and discovering cross-model transferable failures [26], [42]. These requirements capture core attack surfaces and realistic threat assumptions, and DNN fuzzing methods that overlook any of them may provide a misleading sense of security. We formalize these requirements into a unified set of evaluation metrics for assessing DNN fuzzing papers.

We then developed and refined a set of concrete evaluation guidelines to assess DNN fuzzing papers on these metrics, which together form the FuzzCheck.AI codebook (Table II). To initialize the codebook, one author drafted an initial version based on pilot assessments of representative DNN fuzzing papers. For independent validation, a second author applied this initial codebook to the same paper set following established open-coding practices from qualitative research [50]. The two authors iteratively compared assessments, discussed disagreements, refined metric definitions and decision criteria, and re-evaluated papers across multiple rounds. All authors reviewed the revisions, provided feedback, and confirmed consensus before finalizing the codebook.

B. FuzzCheck.AI Metrics and Codebook

1) Failure Severity: Discovering failures in undefended models is often insufficient for security evaluation, because modern DNNs are typically deployed with safety, policy, or security safeguards intended to prevent, detect, or mitigate adversarial and safety-violating behavior [51], [52], [53], [54], [55], [56], [57]. A fuzzer may uncover many failures on an undefended model, but such failures do not necessarily reflect weaknesses that an attacker could exploit in practice [58], [59],

TABLE II
FUZZCHECK.AI CODEBOOK WITH EVALUATION CRITERIA FOR HIGH, MEDIUM, AND LOW FULFILLMENT LEVELS (**C.** = CODE).

High (●)	**Medium (◑)**	**Low (○)**	**Coding Guidance**
Failure Severity (FS)			
Targets a model equipped with explicit safety, policy, or security safeguards (e.g., adversarially trained or safety-aligned models), and empirically demonstrates failures that bypass those defenses.	Discovers harmful, unsafe, biased, or toxic, but does not empirically demonstrate bypass of an explicit safeguard.	Discovers only robustness-related failures (e.g., accuracy drop, mispredictions), without harmful behavior or safeguard bypass.	Use the strongest failure class supported by the fuzzing design and demonstrated in the evaluation. Do not rely only on the paper's motivation, threat discussion, or claimed security relevance.
Targeted Attack Discovery (TA)			
Supports targeted attack discovery by steering exploration toward specific attacker-chosen objectives (e.g., inducing a classifier to produce a specified target label, a speech model to transcribe to a chosen command, or an LLM to generate a response that satisfies the specified malicious objective).	Steers exploration toward broader classes of safety- or security-violating behavior (e.g., bypassing refusal without verifying relevance to the malicious query, or inducing toxic, biased, or unsafe output), without targeting a specific predefined attacker objective.	Performs untargeted exploration toward prediction deviation or inconsistency, without targeting predefined goal or failure class.	For generative models (e.g., LLMs), predefined target outputs are impractical; outputs that satisfy a specific attacker objective (e.g., functional malware, a directly relevant harmful response) suffice for a High rating.
Input Plausibility (IP)			
Enforces semantic consistency (e.g., by fixing semantically critical content such as the target question in a jailbreak template or validating semantic similarity to the original seed) and ensures naturalness (e.g., by validating realism, controlling perturbation relative to original seed, or applying naturalness preserving operations like LLM-based rewriting) across fuzzing iterations.	Enforces semantic consistency and naturalness, each where applicable, between successive mutation steps only (e.g., by controlling similarity or perturbation magnitude relative to the previous mutated input rather than the original seed) without accounting for cumulative distortion across iterations.	Does not enforce semantic consistency or naturalness, allowing unrealistic or semantically invalid inputs to induce failures.	Semantic consistency and naturalness are each assessed only where applicable to the fuzzing design and motivation. Assign N/A only if neither dimension applies.
Failure Reproducibility (FR)			
Explicitly enforces all input format constraints relevant to the target input representation during mutation (e.g., clipping to valid ranges and rounding to integer precision for images, clipping to valid amplitude ranges for audio), or generates inputs directly in a serialized format (e.g., audio synthesized by TTS engine, images generated in standard formats such as PNG or JPEG, text generated by LLM).	Performs mutations directly on raw input values (e.g., rotation/brightness change in image pixels, speed/volume change in audio waveform) without enforcing relevant format constraints, or partially enforces format constraints in continuous intermediate representations (e.g., clipping to the normalized range without ensuring perturbations survive denormalization).	Mutates or synthesizes inputs in continuous intermediate representations (e.g., normalized, frequency domain, latent space, embedding space) without enforcing any relevant format constraints (e.g., no clipping or rounding).	Examine the mutation design and implementation details to determine how input format constraints are handled. Also examine whether the paper discusses or demonstrates failure reproducibility in its evaluation.
Failure Diagnostics (FD)			
Provides diagnostic analysis that explains why failures occur by linking them to underlying model vulnerabilities (e.g., tracing discovered failures to corrupted or biased training data, spurious features, overfitting, or distribution shift)	Provides partial diagnostic insight by characterizing properties of the discovered failures (e.g., class-level error concentration, correlations between failures and input structure), or how they correlate with model-internal signals (e.g., neuron activation patterns).	Reports discovered failures but does not analyze model internals or failure patterns to provide diagnostic insight.	The diagnostic insight must come from analysis of the discovered failures, not from the design rationale or motivation. Hypotheses about potential causes should not be considered as failure diagnostics.
Attack Transferability (AT)			
The fuzzing design incorporates explicit mechanisms to support the discovery of transferable failures across models (e.g., guiding mutation using feedback from surrogate models), and demonstrates cross-model transferability of the discovered failures.	Demonstrates cross-model transferability of the discovered failures, but the fuzzing design does not incorporate explicit mechanisms for transferable attack discovery.	Does not demonstrate whether fault-inducing inputs discovered on one model also trigger failures in other models.	Examine whether fuzzer discovered fault-inducing inputs are applied to separate target models post-hoc to check whether failures carry over. Multi-model disagreement used as a differential oracle during fuzzing is not cross-model transferability.

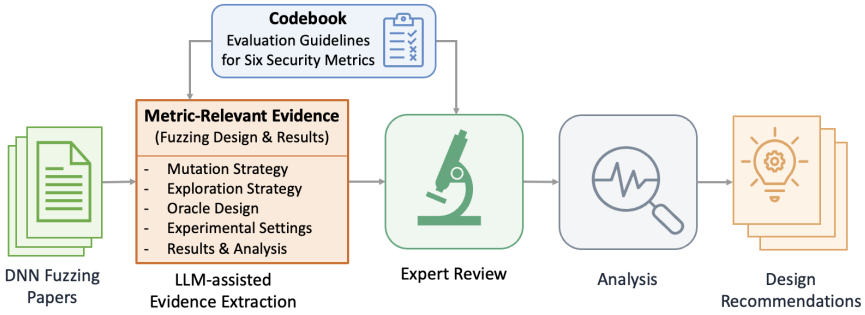

Fig. 1. Overview of FuzzCheck.AI framework.

[60]. Moreover, discovered failures differ in their practical security significance. Failures involving harmful, unsafe, or policy-violating behavior, especially when they bypass explicit safeguards, are more security-relevant than robustness-related errors such as accuracy degradation, mispredictions, or prediction inconsistencies. This metric therefore evaluates the practical severity of the failures that a fuzzer is designed and empirically shown to discover. To assess papers on this metric, we examine the most severe class of failure that is both supported by the fuzzing design and demonstrated in the evaluation, based on the target models, oracle definition, and reported evaluation setting.

2) Targeted Attack Discovery: In many realistic threat models, attackers seek to induce specific attacker-chosen outputs (e.g., unsafe content labeled as safe or speech transcribed to chosen commands) rather than arbitrary model errors [30], [8], [9]. For example, an attacker may be more interested in modifying a malicious binary so that it is classified as benign, rather than causing benign software to be classified as malicious. Similarly, an attacker may seek to induce an LLM to provide assistance that satisfies a specific harmful objective, such as generating functional malware code or step-by-step attack instructions. This metric captures whether and how a DNN fuzzer steers its exploration toward predefined attack goals, distinguishing between fuzzers that aim to induce specific attacker-chosen outputs, broader classes of safety- or security-violating behavior (e.g., jailbreaks, toxicity, or unsafe behaviors), and untargeted faults such as misclassifications or prediction inconsistencies.

3) Input Plausibility: When a fuzzer assumes that mutations should preserve semantics or remain realistic within the task domain, input plausibility becomes critical for two reasons. First, when mutations do not preserve semantics, ground-truth labels become uncertain. Many DNN fuzzers therefore rely on semantic-preserving mutations and oracles that assume semantic equivalence between mutated inputs and their seed inputs [12], [21], [26]. If mutations produce semantically invalid inputs, a changed model output may no longer indicate a meaningful failure, but instead reflect a breakdown of the oracle assumption itself. For instance, if a jailbreak prompt does not retain its semantic meaning after mutation, an absence of refusal in the output becomes irrelevant. Second, in realistic threat models, the security relevance of a discovered failure depends on whether it is induced by inputs that remain realistic within the task domain [7], [61], [62], [21]. For instance, a stop sign that is slightly brightened or rotated remains plausible within the driving domain, whereas one corrupted by heavy random noise or visually incoherent textures does not [12]. Similarly, speech containing mild background hiss remains intelligible and natural, while severely distorted or robotic utterances fall outside realistic speech conditions [8], [63]. Failures induced by unrealistic inputs may still reveal robustness weaknesses, but they are less meaningful for practical security evaluation.

DNN fuzzers typically generate fault-inducing inputs through iterative mutation of seed inputs. Promising mutants, such as those that trigger failures or expand coverage, are added to the seed pool and selected for further mutation in later iterations [21]. Distortions can therefore accumulate across repeated rounds. A single conservative mutation may preserve semantics and naturalness, but repeated mutations can progressively degrade both, eventually yielding inputs that are no longer semantically consistent or realistic. This drift is especially likely when each mutation is constrained only relative to the most recent mutant rather than the original seed, allowing small per-step changes to compound across iterations. Alternatively, some fuzzers avoid drift by keeping semantically critical content fixed and mutating only surrounding components (e.g., mutating the jailbreak prompt template while keeping the target question fixed [26]).

Using this metric, we evaluate whether DNN fuzzers enforce semantic consistency and naturalness throughout fuzzing iterations. Semantic consistency is evaluated only when the fuzzing design requires it, that is, when the oracle assumes semantic equivalence between mutated inputs and their seeds (e.g., metamorphic oracles). When the oracle does not rely on semantic equivalence (e.g., differential oracles that identify failures through disagreement between models), semantic consistency is not applicable. Naturalness is evaluated when the fuzzer aims to produce realistic fault-inducing inputs. Fuzzers that intentionally generate unrealistic inputs to stress-test robustness or explore extreme behaviors fall outside its scope.

4) Failure Reproducibility: Small numerical perturbations introduced during mutation may induce failures that appear only in memory but disappear after saving the mutated input to disk [48]. In DNN fuzzing, mutations are typically applied to in-memory floating-point representations of inputs [10], [20], [48]. If these mutations do not respect the constraints of the target input format, such as image pixel values being integers in the range $[0, 255]$, audio waveform amplitudes lying within $[-1, 1]$, or text being represented as discrete valid token IDs, fault-inducing perturbations may be altered or lost when the input is saved to disk, as standard serialization enforces these constraints through rounding and clipping. For example, a pixel mutated from 254 to 256 will be clipped to 255 during serialization, and a pixel changed from 155.0 to 155.4 will be rounded back to 155, in both cases eliminating the perturbation that triggered the failure. Mutations applied directly to raw input values, such as pixel intensities or waveform amplitudes, are less sensitive to this problem since they operate in the same numerical space as the serialized format. This problem is more severe when mutations are applied to normalized representations (e.g., pixel values scaled to $[0, 1]$ or standardized using dataset-specific statistics), since perturbations introduced in this space may not survive conversion back to the standard format used for storage. Since real-world DNN systems receive inputs in standard serialized formats (e.g., image files, audio recordings, or text), failures that do not survive serialization cannot be reliably reproduced under realistic deployment conditions and are more likely to be false positives than genuine model weaknesses. The same challenges apply to synthesized test inputs. Generative models such as GANs or neural style transfer networks typically output floating-point tensors in continuous or normalized form, which face this reproducibility issue if not explicitly converted to the target format. Inputs generated directly in the target serialized format, such as audio from TTS engines, images generated in standard formats such as PNG or JPEG, or text strings from language models, are inherently stable with respect to serialization. Using this metric, we evaluate whether DNN fuzzers account for input format constraints so that serialization does not remove or alter fault-inducing perturbations.

5) Failure Diagnostics: Discovering failure-inducing inputs is often insufficient for security evaluation, because failure cases alone do not reveal what underlying model weakness is being exploited or how it should be addressed. A fuzzing paper that reports many failures but provides no diagnostic analysis leaves developers without guidance on root causes or priorities. This metric captures whether and how a DNN fuzzing paper provides diagnostic understanding of discovered failures. For instance, if failures are concentrated in a particular output class, this may indicate dataset bias that a developer could address through targeted data collection or rebalancing [64]. If failures correlate with specific input patterns or features, this may suggest reliance on spurious or non-robust features that could be mitigated through adversarial training, data augmentation, or regularization [49]. Using this metric, we distinguish between papers that explain why failures occur by linking them to underlying model vulnerabilities, papers that provide partial insight by characterizing properties of the discovered failures, and papers that report failures without any diagnostic analysis.

6) Attack Transferability: Models trained independently on the same task often exhibit common vulnerabilities, and fault-inducing inputs discovered by fuzzing one model may transfer to others [60], [26]. Such transferability underlies a realistic and widely studied attack setting in which an adversary crafts fault-inducing inputs against a surrogate model and reuses them against an unseen target [65], [66]. Transferable failures are therefore especially important for security evaluation, as they reveal vulnerabilities that extend beyond a single model instance. This metric captures whether and how a DNN fuzzing paper enables the discovery of transferable failures across models. It distinguishes between papers that incorporate explicit design choices to discover transferable failures and demonstrate cross-model transferability, papers that demonstrate transferability by reusing fault-inducing inputs on other models without explicitly designing for it, and papers that evaluate discovered failures only on the model used during fuzzing.

C. Assessment Methodology

To ensure consistent, scalable, and transparent application of the codebook across papers, we design a semi-automated, two-stage assessment workflow that combines LLM-assisted evidence extraction with expert review. This methodology is motivated by prior work showing substantial overlap between LLM-generated and human reviewer feedback on scientific papers [67], as well as the ICLR 2025 LLM-assisted review feedback study, which highlights the potential of carefully designed LLM-based systems to enhance peer review quality at scale [68].

LLM-assisted Evidence Extraction. In the first stage of our assessment workflow, we use an LLM-based preliminary assessment pipeline that we developed to extract metric-relevant supporting evidence and generate a preliminary assessment with accompanying rationale for each paper. The pipeline consists of two independent evaluator LLMs and an arbitrator LLM. Each LLM is instructed to extract and cite supporting evidence from the paper, a practice shown to improve factual accuracy and reduce hallucinations in LLM outputs [69], [70]. When the two evaluator LLMs produce differing preliminary assessments, the arbitrator re-examines the full paper, the codebook criteria, and the evidence extracted by the evaluators to produce an arbitrated assessment with its own rationale grounded in the extracted evidence. We use GPT-5 mini and Gemini 2.5 Pro as evaluator LLMs, and o4-mini as the arbitrator. These models represent two leading LLM families with documented capabilities for structured reasoning and long-context analysis [71], [72], [73]. All LLMs were invoked using default API parameters. Prompt templates and representative outputs, illustrating how the LLM extracts evidence to justify its assessments and how disagreements between evaluators are

resolved by the arbitrator, are shown in Appendix Listings 1 and 2.

Expert Review. In the second stage, two authors independently review and verify all LLM-generated preliminary assessments. Using the supporting evidence extracted by the LLM pipeline, the authors locate and examine the corresponding design descriptions of the proposed fuzzing method (e.g., mutation strategy, exploration strategy, and oracle definition), as well as the reported experimental results. For each metric, they assess whether the extracted evidence is relevant, whether the LLM-generated rationale correctly interprets the paper, and whether the paper satisfies the corresponding codebook criteria. Based on this analysis, the authors assign a final compliance level for each metric. Any disagreements between the two authors are resolved through discussion until consensus is reached, reducing subjective interpretation.

Reliability of LLM-assisted Assessment. Using this assessment workflow, we evaluated 32 DNN fuzzing papers against the six FUZZCHECK.AI security metrics. The two evaluator LLMs disagreed in 44 cases, which were resolved by the arbitrator. In these cases, the arbitrator LLM's assessment aligned with the GPT-5 evaluator in 54.5% of instances and with the Gemini evaluator in 45.5%, indicating no model-family bias. Overall, LLM-based preliminary assessments aligned with expert assessments in 87% of cases, demonstrating their reliability in accelerating reliable evaluation. Expert–LLM assessment comparisons and agreement rates per metric are provided in Appendix Tables VII and VIII, respectively.

V. LITERATURE SELECTION AND TAXONOMY

A. Literature Selection

We conducted a systematic web search across major academic databases, including Google Scholar, IEEE Xplore, ACM Digital Library, USENIX, NDSS, OpenReview, and Semantic Scholar, using a set of relevant keywords. While some DNN fuzzing studies explicitly use the term "fuzzing", others adopt fuzzing-inspired principles under broader descriptions such as "DNN testing" or "deep learning testing". To capture both categories, we constructed search queries by pairing fuzzing/testing terms with various model architectures, tasks, and domains (e.g., machine learning, deep learning, large language models, speech recognition, image classification, and autonomous driving). This approach ensured broad and inclusive coverage of studies aligned with the core fuzzing paradigm, even when not explicitly labeled as such.

Initial search results were screened by title and abstract to exclude irrelevant studies, including those focused on traditional software fuzzing that use AI models as tools (e.g., for input generation) or on fuzzing of deep learning libraries and frameworks. The authors independently screened all retrieved papers for relevance, yielding 115 studies for in-depth full-text review. In the next phase, we examined each paper's methodology and experimental workflow to determine whether it implemented a systematic exploration of the input space, a defining property of fuzzing-based testing. We excluded those whose objectives and workflows differ fundamentally from fuzzing, such as coverage analyses that measure how thoroughly a fixed test dataset explores a model, metamorphic testing that checks semantic consistency under predefined transformations on fixed test dataset, and optimization-based adversarial attacks that aim to find evasive examples rather than systematically probe model behavior. 59 papers were excluded at this stage.

Finally, we filtered papers based on venue reputation and research impact. We included all papers published in CORE-ranked A* and A conferences [74], which represent the leading venues for AI, software testing, and security research. For journal and arXiv papers, we applied a citation threshold of 30 (as of April 2026) to ensure the inclusion of well-established and widely recognized studies. 18 papers were excluded during this step, resulting in a final corpus of 38 DNN fuzzing studies, which we systematically assessed using our FUZZCHECK.AI framework.

B. Taxonomy of DNN Fuzzing Approaches

We organize these studies along four design dimensions: oracle design, mutation strategy, exploration strategy, and access level. These dimensions capture how a fuzzer defines failure conditions, generates test inputs, guides input-space exploration, and how much internal model information it can access during fuzzing. This taxonomy enables a systematic analysis of how design choices influence security testing outcomes. Table III shows how each fuzzer maps onto these design choices.

1) Oracle Design: The oracle determines whether a test input reveals a model fault. In DNN fuzzing, manually labeling ground truth for mutated or synthesized inputs is infeasible, so fuzzers infer correctness using alternative reference signals, such as consistency across semantically equivalent inputs, agreement across multiple models, or violations of explicit safety or task-specific properties. Based on these reference sources, oracle designs fall into three categories:

Metamorphic Oracle. A metamorphic oracle assumes that semantics-preserving input mutations should yield the same model prediction as the original input. Under this design, the fuzzer applies transformations (e.g., rotation, brightness changes, paraphrasing) that preserve input semantics and flags a failure when the model produces inconsistent outputs across these variants. This design is widely used in DNN fuzzing frameworks (e.g., *DLFuzz*, *DeepHunter*, *DeepCruiser*), where violations of such metamorphic relations are treated as model faults [20], [21], [12], [63], [39], [75]. Among the 38 evaluated DNN fuzzers, 17 adopt metamorphic oracles.

Differential Oracle. A differential oracle compares outputs across multiple models performing the same task and flags a failure when the models disagree on the same input. Under this design, the fuzzer executes multiple models on the same mutated input and treats disagreement as evidence of erroneous behavior, without requiring ground-truth labels [10], [40], [25]. However, this approach cannot determine which model is faulty. Among the 38 evaluated DNN fuzzers, 6 adopt differential oracles.

Property-based Oracle. A property-based oracle encodes the failure condition directly in terms of expected model behavior, defining explicit rules over model outputs (e.g., producing a specified output, generating unexpected values, or violating safety or security policies). Under this design, the fuzzer flags a failure when the model output satisfies a failure condition defined by these properties. Unlike metamorphic and differential oracles, which rely on relative comparisons, this approach directly specifies the desired or prohibited behavior. For example, *LLM-Fuzzer* [23] detects jailbreak attacks by checking whether model responses lack expected refusal patterns, while *TensorFuzz* [11] identifies failures by detecting numerical errors (e.g., NaNs/Infs) in model outputs or loss values. Among the 38 evaluated DNN fuzzers, 15 adopt property-based oracles.

2) Mutation Strategy: The mutation strategy defines how a fuzzer modifies existing inputs or generates new ones. We categorize mutation strategies based on how mutations are guided during input generation, resulting in three categories:

Feedback-informed Mutation. Applies perturbations to seed inputs whose magnitude and direction are determined by model feedback (e.g., loss, confidence, predictions, or coverage). This steers mutations toward regions more likely to induce failures. For example, *DeepXplore* [10] perturbs inputs along gradients of a joint objective that maximizes neuron coverage while inducing differential behaviors across models. Among the 38 evaluated DNN fuzzers, only 5 adopt this mutation strategy.

Rule-based Mutation. Modifies seed inputs using predefined rules, templates, or domain-specific heuristics without relying on model feedback. These mutations can be metamorphic, preserving the underlying semantics while changing their appearance or presentation; for example, adjusting brightness or rotation in images, altering playback speed or pitch in audio, or replacing words with synonyms in text [21], [76]. Others intentionally alter content to test robustness under plausible, semantically shifted conditions, such as inserting or removing objects in images or phrases in text [77]. Among the 38 evaluated DNN fuzzers, 22 use rule-based mutation.

Generative Input Synthesis. Instead of modifying seed inputs through perturbations or predefined rules, this strategy synthesizes test inputs using auxiliary generative models. Test inputs may be produced from scratch or conditioned on existing seeds using generative models such as large language models (LLMs), generative adversarial networks (GANs), diffusion models, or text-to-speech (TTS) engines. By leveraging learned data distributions, generative synthesis can produce diverse and realistic inputs. For example, LLMs generate semantically coherent text inputs [22], [26], [23], TTS engines synthesize speech from natural text corpora [40], [78], and GANs render realistic driving scenes under different environmental conditions [79]. Among the 38 evaluated DNN fuzzers, 11 employ generative input synthesis.

3) Exploration Strategy: Exploration strategy refers to the mechanism a DNN fuzzer uses to select and prioritize candidate inputs throughout the fuzzing process. DNN fuzzers guide this search using different sources of feedback, including coverage signals derived from neuron activations, model outputs, and input-domain information. Based on the type of feedback used to guide search, we group exploration strategies into three categories:

Coverage-guided Exploration. Inputs are prioritized based on their ability to expand model coverage. Since DNNs operate in continuous, high-dimensional activation spaces, fuzzers approximate coverage using internal model signals. Common coverage metrics include neuron-activation coverage [10], [15], which measures whether individual neuron outputs exceed or fall within specified activation ranges; neuron-path coverage [80], which measures whether inputs exercise distinct critical decision paths through the network; surprise adequacy [81], which measures how unusual an input is relative to the training data; and layer-wise distributional divergence [39], [43], which measures diversity in activation distributions across layers. Higher model coverage has been shown to correlate with increased fault discovery [39]. Among the 38 DNN fuzzers we evaluate, 16 employ coverage-guided exploration.

Prediction-guided Exploration. This strategy selects inputs using feedback derived from model outputs, signals that estimate how close an input is to inducing failure. Common examples include prediction disagreement or prediction error [10], [20], [48], uncertainty in the model's output [82], estimated failure likelihood based on prior prediction outcomes [40], [25], and cumulative reward reduction induced by the model's decisions in sequential decision-making tasks [83]. LLM fuzzers such as AutoDAN [26], PAIR [22], PAPILLON [42], and LLM-Fuzzer [23] similarly score candidate prompts based on the harmfulness or jailbreak success of the target model's output. Of the 38 DNN fuzzers we evaluate, 22 employ prediction-guided exploration. Of these, 4 fuzzers (DeepXplore [10], DLFuzz [20], TESTRNN [84], and TACTIC [79]) combine prediction and coverage feedback to guide exploration.

Data-driven Exploration. Rather than relying on model feedback, this strategy guides exploration using signals derived from the input domain itself. Among the 38 DNN fuzzers we evaluate, only 4 employ data-driven exploration. LipFuzzer [85] uses offline Bayesian-network linguistic models, built from external linguistic knowledge and user-study data, to generate plausible pronunciation-, vocabulary-, and grammar-level mutations and prune unlikely mutation paths. semSensFuzz [77] explores the input space by randomly sampling mutation candidates that satisfy domain-specific semantic and conformity constraints. QATest [86] prioritizes questions based on statistical and structural properties of the question data, favoring those that increase N-gram coverage of the seed corpus and remain more natural and semantically coherent after transformation. TWINFuzz [87] filters candidate inputs using SSIM, a domain-specific perceptual quality threshold, retaining only those that remain visually similar to their originals.

4) Access Level: The access level defines how much internal visibility a fuzzer has into the target model. Whether it can inspect gradients, neuron activations, and model parameters, or instead operates solely on model outputs, determines the range of feasible mutation strategies and feedback mechanisms. The access level also reflects the underlying threat model, determining what an attacker can realistically observe about the system under test. We distinguish three access levels:

Whitebox. A whitebox fuzzer has full access to model parameters, gradients, and neuron activations. This enables gradient-guided mutation, where gradient signals determine the direction and magnitude of input perturbations toward misclassification or coverage expansion [10], [20], [88]. Among the 38 DNN fuzzers we evaluate, 5 operate in a whitebox setting.

Greybox. A greybox fuzzer has partial access to model internals, typically limited to neuron activations or intermediate layer outputs, but not parameters or gradients. This supports activation-based feedback for coverage-guided exploration [21], [39], [12] but does not support gradient-guided mutation. Among the 38 DNN fuzzers we evaluate, 13 operate in a greybox setting.

Blackbox. A blackbox fuzzer has no visibility into the target model's internals and interacts with the model only through observable outputs. Fuzzing in this setting relies on output-level feedback, such as predicted labels, confidence scores, or generated responses, to guide mutation and exploration. This setting aligns closely with realistic deployment scenarios in which model internals are inaccessible to attackers. For example, LLM fuzzers such as AutoDAN [26], PAIR [22], and LLM-Fuzzer [23] operate in blackbox settings by iteratively refining prompts based on observed jailbreak success or harmfulness of responses. Among the 38 DNN fuzzers we evaluate, 20 operate in blackbox setting.

VI. Analysis and Findings

Our findings, as detailed in Table III, show that none of the 38 DNN fuzzing papers satisfy all six security testing metrics, and 24 exhibit low compliance on at least three. We analyze these results across metrics, design choices, and publication year to identify systematic blind spots in existing fuzzing approaches.

A. Failure Severity

Only 21.1% of DNN fuzzers discover high-severity failures that bypass explicit safety or security safeguards on the target model. A further 10.5% expose medium-severity failures, such as harmful or unsafe behavior, but only on models without such safeguards. The remaining 68.4% discover only low-severity robustness errors, including mispredictions, accuracy degradation, or prediction inconsistencies, without demonstrating bypass of any defense mechanism (Table IV). This distribution suggests that most existing DNN fuzzers remain oriented toward robustness testing rather than security-relevant failure discovery.

TABLE III
Fulfillment levels and design choices of 38 DNN fuzzers. **M**, **E**, and **O** denote Mutation, Exploration, and Oracle, respectively. Metrics (FS, TA, IP, FR, FD, AT) are defined in Table II.

Method/Author	Year	Design			Security Metrics					
		M	E	O	FS	TA	IP	FR	FD	AT
Blackbox										
PAIR [22]	2025	G	P	Pr	●	◑	●	●	○	◑
PAPILLON [42]	2025	G	P	Pr	●	●	●	●	◑	◑
ORFuzz [89]	2025	G	P	Pr	○	◑	●	●	◑	◑
AutoDAN [26]	2024	G	P	Pr	●	●	●	●	○	◑
SneakyPrompt [90]	2024	R	P	Pr	●	●	●	●	◑	○
LLM-Fuzzer [23]	2024	G	P	Pr	●	◑	●	●	○	●
CureFuzz [91]	2024	R	P	Pr	◑	◑	●	●	◑	○
DRFuzz [41]	2023	R	P	Df	○	○	●	○	●	○
ASDF [92]	2023	R	P	Df	○	○	●	●	◑	○
MDPFuzz [83]	2022	R	P	Pr	◑	◑	●	●	◑	○
BET [48]	2022	R	P	Pr	○	●	●	●	○	○
QATest [86]	2022	R	D	M	○	○	◑	●	◑	○
semSensFuzz [77]	2022	R	D	M	○	○	●	◑	◑	○
CAGFuzz [93]	2021	G	C	M	○	○	◑	○	○	○
CrossASR++ [25]	2021	G	P	Df	○	○	●	●	○	○
Asyrofi et al. [78]	2021	G	P	Df	○	○	●	●	○	○
Sensei [94]	2020	R	P	Pr	○	○	●	◑	○	○
KuK [82]	2020	R	P	M	●	○	●	◑	◑	◑
CrossASR [40]	2020	G	P	Df	○	○	●	●	○	○
LipFuzzer [85]	2019	R	D	M	◑	○	●	●	◑	○
Greybox										
Zhou et al. [24]	2025	G	C	Pr	●	◑	◑	●	◑	○
Yuan et al. [95]	2024	R	C	M	○	○	◑	○	◑	○
NLC [39]	2023	R	C	M	○	○	◑	◑	◑	○
DistXplore [43]	2023	R	C	Pr	○	●	◑	◑	●	○
TESTRNN [84]	2021	R	C, P	M	○	○	●	◑	◑	○
TACTIC [79]	2021	G	C, P	M	○	○	●	○	◑	○
HDTest [96]	2021	R	P	M	○	○	●	◑	◑	○
DeepHunter [21]	2019	R	C	M	○	○	●	◑	◑	○
TensorFuzz [11]	2019	R	C	Pr	○	◑	●	◑	◑	○
DeepStellar [76]	2019	R	C	Pr	○	○	◑	◑	◑	○
DeepEvolution [97]	2019	R	C	M	○	○	●	◑	○	○
DeepTest [12]	2018	R	C	M	◑	○	◑	◑	◑	○
DeepCruiser [63]	2018	R	C	M	○	○	●	◑	◑	○
Whitebox										
TWINFUZZ [87]	2026	F	D	Pr	●	○	●	○	○	○
RobOT [98]	2021	F	P	M	○	○	◑	○	◑	○
ADAPT [88]	2020	F	C	M	○	○	●	○	◑	○
DLFuzz [20]	2018	F	C, P	M	○	○	●	○	○	○
DeepXplore [10]	2017	F	C, P	Df	○	○	◑	●	●	○

Symbols: ●=High, ◑=Medium, ○=Low.
Mutation: **F**=Feedback-informed, **R**=Rule-based, **G**=Generative Synthesized.
Exploration: **C**=Coverage-guided, **P**=Prediction-guided, **D**=Data-driven.
Oracle: **M**=Metamorphic, **Df**=Differential, **Pr**=Property-based.

Fuzzers rated high on the failure severity metric are concentrated in a relatively narrow region of the design space. In our corpus, fuzzers evaluated on safeguarded models cluster around blackbox access (30% high), generative-synthesis-based mutation (45% high), prediction-guided exploration (27% high), and property-based oracles (47% high) (Table V). Representative examples include PAIR [22], PAPILLON [42], AutoDAN [26], and LLM-Fuzzer [23], which discover jailbreak attacks on safety-aligned LLMs such as GPT-4 and Gemini Pro. SneakyPrompt [90] discovers jailbreak prompts in safeguarded text-to-image generative models, bypassing the closed-box safety filter of DALL·E 2 to generate prohibited NSFW images. This concentration is consistent with both deployment constraints and the security-relevant failure conditions these fuzzers are designed to trigger. Most of these fuzzers target commercially deployed safety-aligned LLMs, which are typically accessible only through limited API interfaces, making blackbox access the dominant realistic setting. Beyond this access constraint, the goal of eliciting harmful or prohibited outputs makes property-based oracles a natural fit, since they can encode policy-violating behavior directly in the failure condition. Likewise, generative-synthesis-based mutation, such as LLM-based prompt rewriting, is well suited to text-input models, where semantically coherent prompt rewriting helps preserve fluency while probing for safety bypasses.

In contrast, fuzzers with richer instrumentation and more granular feedback, such as whitebox or greybox access, feedback-informed mutation, and coverage-guided exploration, have rarely been evaluated on safeguarded models. Among the five whitebox fuzzers in our corpus, only TWINFUZZ [87] is evaluated on adversarially trained robust models, achieving high severity with feedback-informed mutation. Among greybox fuzzers, only Zhou et al. [24] include a safety-aligned model, Llama-2-7B-Chat, in their evaluation scope and leverage coverage-guided exploration. KuK [82] is the only case in which a metamorphic oracle achieves high severity by discovering failures that bypass adversarial attack detection techniques. No fuzzer using a differential oracle has been evaluated on a safeguarded model (0 of 6; 100% low). This gap appears to reflect current evaluation practice rather than an inherent limitation of differential oracles. In principle, a differential oracle could be applied to safeguarded models by identifying prompts for which one model refuses while another complies, thereby exposing jailbreak-style failures.

Fuzzers rated medium severity form a distinct cluster. All four use rule-based mutation and target safety-critical control or autonomous systems. CureFuzz [91] and MDPFuzz [83], for instance, expose unsafe control behaviors in autonomous systems, such as sudden turns, off-route maneuvers, or collision-prone trajectories. LipFuzzer [85] triggers misinterpretations in voice assistants, causing benign commands to launch unintended apps or share data with untrusted recipients. DeepTest [12] exposes dangerous steering deviations under different weather conditions in Udacity challenge models. This pattern indicates that safety-critical control systems can exhibit directly observable unsafe behavior, but are often evaluated without explicit safeguard mechanisms whose bypass would elevate such failures to high severity.

TABLE IV
DISTRIBUTION OF DNN FUZZING STUDIES BY COMPLIANCE LEVEL ACROSS FUZZCHECK.AI METRICS DEFINED IN TABLE II.

Level	FS	TA	IP	FR	FD	AT
High (●)	21.1%	13.2%	73.7%	44.7%	7.9%	2.6%
Medium (◑)	10.5%	18.4%	26.3%	34.2%	60.5%	13.2%
Low (○)	68.4%	68.4%	0.0%	21.1%	31.6%	84.2%

Temporal trends (Table VI) show that DNN fuzzers achieving high fulfillment on the failure severity metric are strongly concentrated in 2024–2026 (7 of 8 papers), with KuK [82] (2020) as the sole earlier exception. This indicates that high severity failure discovery using DNN fuzzing is an emerging focus, not a long-standing practice.

B. Targeted Attack Discovery

Only 13.2% (5 out of 38) of DNN fuzzers achieve high fulfillment on the targeted attack discovery metric by steering exploration toward specified targeted failures, while 18.4% achieve medium and the majority (68.4%) focus on untargeted failure discovery (Table IV). Untargeted fuzzing dominated the literature from 2017 through 2021, with targeted attack discovery emerging primarily in 2022–2025 (Table VI).

Targeted fuzzers are concentrated in a narrow region of the design space, most often associated with property-based oracles, prediction-guided exploration, and generative-synthesis-based mutation (Table V). Targeted attack discovery requires the oracle to verify whether a discovered failure satisfies a specific attacker objective, and property-based oracles are the only oracle type that can explicitly encode such conditions. In our evaluation, 33% of fuzzers using property-based oracles achieve high ratings on this metric. AutoDAN [26], for instance, steers exploration toward outputs containing predefined affirmative prefixes (e.g., "Sure, here is how to ...") and uses an LLM judge to verify that the response answers the malicious query. SneakyPrompt [90] similarly performs goal-directed exploration to generate prohibited NSFW images that remain semantically aligned with the malicious prompt. In contrast, none of the fuzzers using metamorphic or differential oracles achieve high ratings. Metamorphic oracles check for prediction inconsistency across semantically equivalent inputs, while differential oracles detect disagreement across models; neither directly encodes what constitutes a targeted failure. Exploration strategy plays a complementary role: prediction-guided and coverage-guided strategies leverage model feedback to prioritize inputs that move outputs closer to a target condition, whereas data-driven exploration does not incorporate such feedback and therefore offers limited support for steering search toward specific target outputs.

Fuzzers rated medium on this metric steer exploration toward broader classes of safety- or security-violating behavior rather than specific attacker objectives. CureFuzz [91] and MDPFuzz [83], for example, guide exploration toward crash-

triggering or collision scenarios without encoding a specific attacker-chosen end state. Several LLM fuzzers, including PAIR [22], LLM-Fuzzer [23], and Zhou *et al.* [24], discover jailbreak prompts that bypass refusal or elicit harmful responses, but their oracles rely on coarse success indicators such as harmfulness or non-refusal rather than verifying whether the response actually satisfies the malicious query. As a result, these fuzzers can expose broad unsafe behavior while still falling short of true targeted attack discovery, since an LLM may avoid refusal yet return off-topic or vague output that does not satisfy the malicious query.

A particularly notable gap emerges in whitebox fuzzing and feedback-informed mutation. Whitebox access exposes gradients and internal activations that are, in principle, well suited for directional exploration, and feedback-informed mutation can exploit these signals to modify inputs in directions that move model outputs toward a target failure condition. Yet none of the fuzzers with whitebox access or feedback-informed mutation in our corpus achieve high fulfillment on this metric. This is not because whitebox access or feedback-informed mutation is inherently unsuited to targeted attack discovery, but because existing fuzzers pair these capabilities with metamorphic or differential oracles that do not encode target conditions. BET [48], by contrast, shows that targeted failure discovery is achievable even with rule-based mutation, as it defines failure condition using a specified target label and applies a label-specific objective function to iteratively mutate inputs toward the target output. This suggests that combining whitebox, feedback-informed mutation with target-aware oracle design represents an underexplored design point—one that could enable efficient targeted attack discovery and provide a stronger basis for pre-deployment hardening against realistic adversarial objectives.

Overall, these findings indicate that targeted attack discovery primarily depends on oracle design to encode target failure conditions, while prediction- or coverage-guided exploration enables effective search toward targeted failures. Notably, feedback-informed mutation strategies have not been used for targeted attack discovery in existing fuzzers, despite their potential, highlighting an important gap.

C. Input Plausibility

D. Failure Reproducibility

Failure reproducibility appears relatively well supported overall, with 44.7% (17/38) of fuzzers rated high, 34.2% (13/38) medium, and 21.1% (8/38) low (Table IV). A closer examination, however, shows that high and medium fulfillment often arises as a byproduct of input modality and mutation mechanism rather than from deliberate reproducibility-aware design. Only a small subset of fuzzers explicitly enforce the input format constraints needed to ensure failure reproducibility. Year-wise trends (Table VI) show no clear improvement over time, indicating that reproducibility has not received sustained attention as a design priority.

High failure reproducibility is concentrated among fuzzers using generative-synthesis-based mutation. Table V shows that 82% of such fuzzers receive a high rating on this metric. In particular, fuzzers that use LLMs to generate text inputs (e.g., AutoDAN [26], PAIR [22], LLM-Fuzzer [23], PAPILLON [42]) or TTS engines to synthesize audio test inputs (e.g., CrossASR [40], CrossASR++ [25]) produce test cases directly in the serialized form consumed by deployed models. As a result, they avoid the floating-point perturbation and format-conversion issues that can cause failures to disappear when inputs are saved and reloaded. The same advantage also appears in several fuzzers with rule-based mutation.

TABLE V
PERCENTAGE (%) OF STUDIES PER DESIGN CHOICE EXHIBITING DIFFERENT MATURITY LEVELS ACROSS FUZZCHECK.AI METRICS. SHADED CELLS INDICATE WHERE OVER 50% OF STUDIES FALL INTO THAT LEVEL. **L.**=FULFILLMENT LEVEL.

Metric	L.	Access			Mutation			Exploration			Oracle		
		W	G	B	F	R	G	C	P	D	M	Df	Pr
Failure Severity	●	20	8	30	20	9	45	6	27	25	6	0	47
	◑	0	8	15	0	18	0	6	9	25	12	0	13
	○	80	85	55	80	73	55	88	64	50	82	100	40
Targeted Attack Discovery	●	0	8	20	0	14	18	6	18	0	0	0	33
	◑	0	15	25	0	14	36	12	23	0	0	0	47
	○	100	77	55	100	73	45	81	59	100	100	100	20
Input Plausibility	●	60	54	90	60	73	82	50	91	75	65	83	80
	◑	40	46	10	40	27	18	50	9	25	35	17	20
	○	0	0	0	0	0	0	0	0	0	0	0	0
Failure Reproducibility	●	20	8	75	20	32	82	12	64	50	12	83	67
	◑	0	77	15	0	59	0	56	18	25	53	0	27
	○	80	15	10	80	9	18	31	18	25	35	17	7
Failure Diagnostics	●	20	8	5	20	9	0	12	9	0	0	33	7
	◑	40	85	50	40	77	36	69	50	75	82	17	53
	○	40	8	45	40	14	64	19	41	25	18	50	40
Attack Transferability	●	0	0	5	0	0	9	0	5	0	0	0	7
	◑	0	0	25	0	5	36	0	23	0	6	0	27
	○	100	100	70	100	95	55	100	73	100	94	100	67

Symbols: ●=High, ◑=Medium, ○=Low.
Abbreviations: **W**=Whitebox, **G**=Greybox, **B**=Blackbox, **F**=Feedback-informed, **R**=Rule-based, **G**=Generative Synthesized, **C**=Coverage-guided, **P**=Prediction-guided, **D**=Data-driven, **M**=Metamorphic, **Df**=Differential, **Pr**=Property-based.

TABLE VI
YEAR-WISE MATURITY TRENDS OF FUZZCHECK.AI METRICS IN DNN FUZZING STUDIES (2017–2026). CELLS SHOW THE PERCENTAGE (%) OF STUDIES AT EACH COMPLIANCE LEVEL; SHADED CELLS INDICATES DOMINANCE (>50%). **L.**=FULFILLMENT LEVEL.

Metric	L.	'17	'18	'19	'20	'21	'22	'23	'24	'25	'26
Failure Severity	●	0	0	0	25	0	0	0	60	75	100
	◑	0	33	20	0	0	25	0	20	0	0
	○	100	67	80	75	100	75	100	20	25	0
Targeted Attack Discovery	●	0	0	0	0	0	25	25	40	25	0
	◑	0	0	20	0	0	25	0	40	75	0
	○	100	100	80	100	100	50	75	20	0	100
Input Plausibility	●	0	67	80	100	71	75	50	80	75	100
	◑	100	33	20	0	29	25	50	20	25	0
	○	0	0	0	0	0	0	0	0	0	0
Failure Reproducibility	●	100	0	20	25	29	75	25	80	100	0
	◑	0	67	80	50	29	25	50	0	0	0
	○	0	33	0	25	43	0	25	20	0	100
Failure Diagnostics	●	100	0	0	0	0	0	50	0	0	0
	◑	0	67	80	50	57	75	50	60	75	0
	○	0	33	20	50	43	25	0	40	25	100
Attack Transferability	●	0	0	0	0	0	0	0	20	0	0
	◑	0	0	0	25	0	0	0	20	75	0
	○	100	100	100	75	100	100	100	60	25	100

Symbols: ●=High, ◑=Medium, ○=Low.

ASDF [92] and LipFuzzer [85] mutate text and then synthesize audio via TTS, while QATest [86] and SneakyPrompt [90] mutate text directly, again producing inputs already in the form consumed by the target model. For these fuzzers, high reproducibility is a byproduct of operating directly on serialized or discrete input representations rather than the result of explicit reproducibility-aware mutation design.

By contrast, only four fuzzers achieve high reproducibility through explicit design choices. DeepXplore [10] clips pixel values to the valid image range and rounds gradient updates to integer precision. BET [48] applies integer perturbations directly to image pixel bytes and reports a 0.0% inactivation rate, meaning all discovered failures persist after serialization. CureFuzz [91] and MDPFuzz [83] achieve high reproducibility by constraining mutations within physically valid bounds and validating each mutated state through autonomous-agent simulation.

Among fuzzers rated medium on failure reproducibility, the majority (59%) use rule-based mutation (Table V). Most of these fuzzers (e.g., DeepTest [12], DeepCruiser [63], DeepHunter [21], NLC [39]) apply mutations directly to raw input values but do not clip or round the mutated values to enforce target-format constraints. Their medium fulfillment is not because of explicit reproducibility-aware design, but largely a byproduct of operating in the raw input space. Transformations such as brightness, contrast, rotation, and noise on image pixels (or speed and volume changes on audio waveforms) naturally act in the same numerical space as the serialized format. As a result, enforcing the relevant constraints would be straightforward, yet this step is often overlooked. TensorFuzz [11] and TESTRNN [84] partially account for reproducibility by clipping inputs to valid numeric ranges in normalized form, but they still do not ensure that perturbations survive denormalization when inputs are saved and reloaded. Fuzzers rated low on this metric (e.g., DLFuzz [20], RobOT [98], TWINFUZZ [87], CAGFuzz [93]) mutate or synthesize inputs in continuous normalized or latent representations without enforcing any format constraints, leaving fault-inducing perturbations at risk of being silently lost during denormalization or serialization.

E. Failure Diagnostics

Failure diagnostics remain limited in current DNN fuzzing research. Only 7.9% (3/38) of papers provide full failure diagnostics, whereas 60.5% provide only partial diagnostics and 31.6% provide no diagnostic insight at all into discovered failures (Table V). Most papers therefore stop at failure discovery or correlational characterization rather than explaining the underlying model vulnerability exposed by the discovered failures. This gap has persisted over time, with partial diagnostics consistently dominating (Table VI).

The three papers rated high on this metric demonstrate that actionable failure diagnosis is feasible. DeepXplore [10] traces discovered failures back to corrupted training data, correctly identifying 95.6% of mislabeled examples in a data-poisoning setting. DistXplore [43] attributes failures to distribution shift, showing that discovered failures become statistically closer, in logit space, to the distribution of another class. DRFuzz [41] relates concentrated regression failures to class-specific overfitting and bias associated with the retraining process, supported by class-level fault analysis and Grad-CAM saliency inspection.

By contrast, papers with partial diagnostics stop at describing where failures concentrate rather than explaining why the model is vulnerable. A common pattern, especially among greybox and coverage-guided fuzzers (Table V), is to relate discovered failures to model-internal signals such as neuron coverage, hidden-state behavior, or state-transition traces. Because these methods already rely on internal execution signals to guide exploration, they naturally reuse those same signals to characterize failure concentration. ADAPT [88], for example, correlates failures with neuron characteristics such as activation frequency, layer type, and position. These analyses identify where failures concentrate in the model's internal behavior, but not what underlying weakness makes those regions failure-prone. A second pattern is to characterize properties of the failure-triggering inputs themselves. For example, ASDF [92] performs phoneme-level analysis to associate specific sounds with transcription errors, while LipFuzzer [85] links semantic misinterpretations to linguistic ambiguity. Such analyses reveal what kinds of inputs trigger failures, but not what underlying property of the model makes those inputs problematic.

Most current fuzzers are designed and evaluated to maximize failure discovery using proxy signals such as coverage, disagreement, or output inconsistency. Those signals are effective for exposing faults, but weak for explaining why the model is vulnerable. As a result, failure count becomes the dominant basis for comparing fuzzing designs. Without failure diagnosis, it remains unknown which fuzzing designs are better at exposing which model vulnerabilities, a gap that limits the security utility of discovered failures.

F. Attack Transferability

Among the 38 evaluated DNN fuzzing papers, only LLM-Fuzzer [23] explicitly incorporates mechanisms to facilitate the discovery of transferable attacks and empirically demonstrates cross-model transferability. In contrast, 13.2% (5/38) of the papers assess transferability only in post hoc evaluation, showing that some discovered failures transfer to other models for the same task despite lacking any transferability-oriented design mechanisms. The remaining 84.2% (32/38) do not consider transferability as a DNN security testing objective at all (TableIV).

LLM-Fuzzer[23] assigns lower or zero reward to mutated jailbreak templates that bypass only one or two LLMs, thereby steering exploration toward templates that are effective across multiple models. The paper then evaluates the generated templates on unseen open-source and commercial LLMs and reports substantial cross-model transferability. By contrast, AutoDAN[26], PAPILLON[**?**], and PAIR[**?**] test whether jailbreak prompts discovered by fuzzing one LLM can also

jailbreak other LLMs, but their fuzzing designs do not include mechanisms specifically intended to promote the discovery of transferable failures. Similarly, ORFuzz[**?**] evaluates whether over-refusal-inducing benign prompts discovered by fuzzing one LLM induce the same behavior in other LLMs. These fuzzers are motivated by the assumption that LLMs trained on large-scale Internet data may share vulnerabilities. KuK[**?**], in contrast, tests whether failures found on an undefended model remain effective against defended variants.

The remaining 32 fuzzers do not evaluate cross-model transferability at all, largely because their designs are centered on exposing failures in a single model instance rather than discovering fault-inducing inputs that generalize across models. For example, feedback-informed mutation often relies on model-specific internal signals, which naturally biases the search toward single-model failures. A key challenge for such designs is how to incorporate feedback that reflects behavior across multiple models. Importantly, transferable attacks are not unique to LLMs. Transfer learning is widely used in vision, speech, and NLP, meaning that failures discovered on one fine-tuned variant may also propagate to others derived from the same pretrained backbone. The absence of transferability-oriented design outside the LLM setting therefore represents a significant gap in current DNN fuzzing research.

G. Key Findings

Limited support for high-severity failure discovery. Only 18.8% of evaluated fuzzers discover high-severity failures that bypass explicit safety or security mechanisms, while 65.6% focus exclusively on low-severity robustness errors (e.g., prediction inconsistencies under semantically equivalent inputs).

Limited support for targeted attack discovery. Only 9.4% of evaluated fuzzers support targeted attack discovery by steering exploration toward attacker-chosen outputs, while the majority (68.8%) prioritize untargeted failure discovery, leaving the discovery of targeted security-violating behaviors largely unexplored.

Overlooked cumulative mutation effects. Although the majority (62.5%) of evaluated DNN fuzzers achieve high input plausibility, largely by leveraging generative synthesis-based mutation to produce realistic test inputs (91%), a substantial fraction (37.5%) fail to account for cumulative mutation effects across iterations, leading to the discovery of failures induced by unrealistic or semantically invalid inputs.

A recurring blind spot in failure reproducibility. Over half (53.1%) of evaluated fuzzers overlook serialization effects (e.g., clipping, rounding), resulting in fragile in-memory failures that may not persist after standard I/O operations.

Near-absence of causal failure diagnostics. Most DNN fuzzing papers (96.8%) provide only partial or no failure diagnostics: 65.6% offer correlational insights based on model internals or failure statistics, while 31.2% provide no diagnostic analysis.

Rare support for transferable attack discovery. Only one DNN fuzzer is explicitly designed to discover transferable failures. Although a small fraction (12.5%) evaluates transferability after discovery, most fuzzers do not incorporate mechanisms to actively discover failures that transfer across models, limiting insight into shared model vulnerabilities.

Fragmented support for comprehensive security testing. No evaluated fuzzer achieves high compliance across all six metrics, and the majority (59.4%, 19/32) exhibit low compliance on three or more metrics, indicating that most fuzzers are designed to address isolated requirements rather than comprehensive security testing.

VII. Future Research Directions

High-severity failure discovery. To enable high-severity security failure discovery, we recommend using models deployed with defense mechanisms as fuzzing targets, such as adversarially trained or safety-aligned models, or models protected by external filtering or detection components. Mutation and exploration strategies in fuzzing design can incorporate feedback signals from the defense mechanisms. Security and policy violations (e.g., jailbreaks, hallucinations, data leakage, or backdoor trigger activation) should be explicitly encoded in the oracle design to discover high-severity security failures.

Targeted attack discovery. To support targeted attack discovery, we recommend using target-aware exploration strategies that prioritize seeds whose outputs are closer to attacker-chosen targets. We further recommend mutation strategies that enable directional control toward target model outputs. Feedback-informed mutation strategies, especially gradient-guided mutations, are well suited for targeted attack discovery because they can provide explicit mutation direction toward inducing attacker-chosen outputs. In contrast, rule-based and generative-synthesis mutation strategies typically lack directional control, leaving exploration alone to guide progress toward the target. Together, these design choices can transform DNN fuzzing from untargeted robustness testing into systematic discovery of targeted security violations.

Input plausibility enforcement. To ensure the plausibility of discovered fault-inducing inputs, we recommend that future DNN fuzzers account for cumulative mutation effects by tracking each mutated input relative to its original seed and discarding candidates that drift beyond a predefined threshold. Such drift can be quantified using task-specific measures, including LPIPS [99] or SSIM [100] for visual similarity, PESQ [101] or STOI [102] for speech quality and intelligibility, and perplexity [103] for linguistic fluency. Alternatively, plausibility can be enforced using an auxiliary model that learns the distribution of valid inputs for the target domain and filters out mutated inputs that fall outside this distribution.

Failure reproducibility enforcement. To address failure reproducibility, we recommend adopting reproducibility-aware mutation designs that explicitly account for serialization effects. In particular, mutation operators should enforce clipping and rounding so that model outputs produced by mutated inputs remain consistent before and after standard I/O operations. For image inputs, this includes clipping pixel values

to valid ranges and rounding to integer precision. For normalized representations, mutations should be constrained such that de-normalization yields valid pixel values. Similarly, for audio inputs, mutated waveforms should be clipped to valid amplitude ranges to ensure that discovered failures persist after serialization.

Causal failure diagnostics. To improve failure diagnostics, we recommend evaluating fuzzers on target models with known vulnerabilities (e.g., reliance on spurious features, dataset bias, or overfitting) and analyzing whether discovered failures systematically correlate with these vulnerabilities. For example, if a model is known to rely on spurious correlations for specific classes, a concentration of failures on those classes provides evidence that the fuzzer exposes causally meaningful security failures rather than incidental misbehavior.

Transferable attack discovery. To enable transferable attack discovery, we recommend designing fuzzers that incorporate feedback from multiple models on the same task. In particular, mutation and exploration strategies can leverage cross-model signals, such as prediction changes, confidence shifts, or satisfaction of failure conditions across models, to guide test generation toward discovery of transferable failures.

Integrating complementary strengths. We recommend integrating fuzzing designs with complementary strengths to jointly satisfy all security testing requirements. For example, DeepHunter [21] is a greybox fuzzer based on metamorphic mutation and coverage-guided exploration. While effective for DNN robustness testing, it does not support high-severity failure discovery, targeted attack discovery, or transferable attack discovery, and only partially supports failure reproducibility and failure diagnostics. To improve its compliance across all metrics, several complementary design strategies can be integrated. For targeted attack discovery, mutation operator selection can be guided by feedback indicating how specific transformations (e.g., translation, rotation, or brightness adjustment) move model outputs toward attacker-chosen targets. For transferable failure discovery, feedback from multiple models performing the same task can be incorporated to prioritize mutation operators and seed inputs that induce failures across models. Correspondingly, oracle designs should encode attacker-chosen targets and cross-model consistency to identify targeted and transferable failures. For high-severity failure discovery, feedback from deployed defense mechanisms (e.g., JPEG compression [57]) can be leveraged. Finally, high compliance on failure diagnostics and failure reproducibility can be achieved by incorporating the metric-specific mechanisms outlined in the preceding recommendations.

VIII. Reproducibility and Extensibility

Our evaluation framework is designed to facilitate transparent, reproducible, and scalable evaluation of DNN fuzzing approaches. Its clear definition of evaluation metrics and structured codebook enables researchers and stakeholders to accurately replicate our study. Importantly, the DNN security testing requirements captured by FuzzCheck.AI extend to other DNN testing paradigms, including adversarial attack-based testing [104], metamorphic testing [13], and emerging evaluation methods. New dimensions can be incorporated by expanding the codebook without modifying the assessment workflow, enabling systematic adaptation to evolving security testing needs.

future fuzzing design - we can input the plan of fuzzing design components to FuzzCheck.AI to see how that supports these secuirty metrics.

IX. Discussion

Limitations. Our evaluation is limited by the completeness and clarity of methodology descriptions and experimental results reported in DNN fuzzing papers. As the analyzed works are selected from peer-reviewed, reputable venues, we assume that their reported designs and results are accurate. Since the assessment of research papers is inherently subjective, we employ a structured codebook, LLM-assisted evidence extraction, and independent expert review to promote consistency and reduce bias. LLMs support metric-related evidence extraction and preliminary assessment, but final assessments are based on independent expert review, with disagreements among experts resolved through discussion. These safeguards mitigate both LLM hallucinations and human subjectivity. Although LLM behavior may evolve with model updates, transparent evidence citation and expert verification help ensure reproducibility and interpretability.

Future Work. An important direction for future work is to translate FuzzCheck.AI into a quantitative evaluation framework that enables standardized benchmarking of DNN fuzzing methods across all six metrics. Such a framework could include models deployed with explicit defense mechanisms, such as adversarially trained or safety-aligned systems, as fuzzing targets, enabling controlled evaluation of a fuzzer's ability to discover high-severity security failures. It could further assess targeted attack discovery capability by quantifying a fuzzer's ability to generate inputs that induce attacker-chosen outcomes, such as malware classified as benign, unsafe images labeled as safe, or benign speech transcribed as unauthorized commands. The framework could also quantify input plausibility by measuring the degree to which fault-inducing inputs preserve realism and semantic validity, using domain-specific metrics such as LPIPS or SSIM for images, PESQ or STOI for speech, and perplexity for text. Failure reproducibility could be evaluated by quantifying the proportion of discovered failures that persist after fault-inducing inputs are stored to disk and reloaded. Failure diagnostics could be assessed by incorporating target models with known vulnerabilities and measuring how discovered failures correlate with these weaknesses, for example by examining whether failures concentrate on classes affected by spurious correlations, dataset bias, or overfitting. Finally, attack transferability could be measured by quantifying the proportion of fault-inducing inputs that generalize to other models performing the same task across different architectures and implementations.

X. Conclusion

This study reveals systematic blind spots in current DNN fuzzing designs across core security testing requirements. Applying FuzzCheck.AI to 32 representative DNN fuzzing papers, we find that no existing fuzzer achieves high compliance across all metrics, with limited support for security-relevant failure discovery (e.g., high-severity, targeted, and transferable attacks), blind spots in input plausibility and failure reproducibility, and a near absence of causal failure diagnostics. These findings highlight a central open challenge: designing DNN fuzzers that efficiently discover security failures while jointly satisfying all security testing requirements. Our recommendations outline concrete guidelines for achieving high compliance on each metric and for integrating fuzzing designs with complementary strengths to enable comprehensive DNN security testing.

Ethical Considerations

This work is a Systematization of Knowledge (SoK) that analyzes DNN fuzzing research based on publicly available, peer-reviewed academic papers. The study does not involve experimentation on live systems, interaction with deployed models, collection of user data, or use of proprietary datasets. All assessments are derived from evidence explicitly reported in the evaluated literature.

Stakeholder Analysis. This study involves four stakeholder groups. (1) *Authors of evaluated DNN fuzzing papers:* Their methods, design choices, and reported empirical results are interpreted and categorized using a structured evaluation framework. (2) *AI security research community:* Researchers and practitioners may rely on the synthesized findings to guide future fuzzing designs, evaluation practices, and research directions. (3) *Research team:* During the assessment process, the researchers of this work may be exposed to potentially harmful model inputs, such as adversarial examples or jailbreak prompts reported in the evaluated papers. (4) *Adversaries:* Although not direct participants, adversaries may gain insights into common limitations or under-explored testing dimensions across fuzzing approaches.

Impact of the Research. The publication of FuzzCheck.AI has both positive and negative impacts on the identified stakeholders.

Positive Impacts. This work (1) provides the AI security research community with a structured and transparent evaluation framework for assessing the security testing capabilities and limitations of DNN fuzzing approaches; (2) reveals recurring blind spots and trade-offs trade-offs among fuzzing design choices and DNN security testing metrics; and (3) informs future DNN fuzzing design by distilling actionable recommendations to improve security failure discovery.

Negative Impacts. (1) Assessments may be inaccurate for some evaluated papers due to incomplete reporting, ambiguous descriptions, or subjective interpretation, potentially misrepresenting reported methods or empirical evidence. (2) The research team may encounter descriptions of potentially harmful model inputs, such as adversarial examples or jailbreak prompts, reported in the evaluated papers. (3) Aggregated insights may indirectly inform adversaries about under-explored testing dimensions or common limitations across classes of fuzzing approaches.

Mitigations. (1) We mitigate the risk of potential inaccurate assessments by grounding all evaluations in a transparent codebook with explicit criteria. Each assessment is supported by evidence from the evaluated paper. All LLM-extracted evidence and preliminary assessments are independently reviewed by two authors, with disagreements resolved through discussion and consensus. (2) Encounter to potentially harmful model inputs reported in the evaluated papers is mitigated by limiting analysis to descriptive content in the literature and avoiding the reproduction, replay, or execution of adversarial examples or jailbreak prompts. (3) Any potential adversarial misuse is limited by the scope of this work, which reports aggregated observations derived from existing literature and does not introduce new attack methods or exploit strategies.

Justification for Research. This work addresses the lack of clarity in DNN fuzzing research regarding how different fuzzing approaches satisfy core security testing requirements. By systematically evaluating representative fuzzing approaches against a structured set of security-oriented metrics, the study reveals recurring blind spots and trade-offs in current practices and distills concrete recommendations for improving security failure discovery. We determined that the benefits of this work outweigh the risks identified above, which are mitigated through methodological safeguards.

Open Science

We provide the following artifacts to facilitate independent verification of our results. All materials are hosted in an anonymized repository: https://anonymous.4open.science/r/FuzzCheckAI-18A2/

- **FuzzCheck.AI Codebook.** The rubric defining criteria for the three compliance levels for each FuzzCheck.AI metric.
- **LLM-Assisted Preliminary Assessment Pipeline.** Implementation of the LLM-assisted assessment workflow, including evaluator and arbitrator stages.
- **LLM-Generated Preliminary Assessments.** Preliminary assessments generated by the LLM pipeline for all 32 DNN fuzzing papers, including extracted evidence and rationales.
- **Expert-Validated and Corrected Final Assessments.** Final assessments after expert review, including documented corrections and rationales where LLMs misinterpreted evidence or extracted incorrect evidence.
- **Analysis Scripts.** Scripts used to generate all summary tables in this paper from the final validated assessments.

Upon acceptance, we will replace the anonymized repository with a stable, non-anonymous repository to ensure continued public availability.

References

[1] B. Vidgen, A. Harris, D. Nguyen, R. Tromble, S. Hale, and H. Margetts, "Challenges and frontiers in abusive content detection," in *Proceedings*

of the third workshop on abusive language online. Association for Computational Linguistics, 2019.

[2] M. B. Hoy, "Alexa, siri, cortana, and more: an introduction to voice assistants," *Medical reference services quarterly*, vol. 37, no. 1, pp. 81–88, 2018.

[3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[4] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," *arXiv preprint arXiv:1606.06565*, 2016.

[5] D. Heaven *et al.*, "Why deep-learning ais are so easy to fool," *Nature*, vol. 574, no. 7777, pp. 163–166, 2019.

[6] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.

[7] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[8] N. Carlini and D. Wagner, "Audio adversarial examples: Targeted attacks on speech-to-text," in *2018 IEEE security and privacy workshops (SPW)*. IEEE, 2018, pp. 1–7.

[9] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does llm safety training fail?" *Advances in Neural Information Processing Systems*, vol. 36, pp. 80 079–80 110, 2023.

[10] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated whitebox testing of deep learning systems," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 1–18.

[11] A. Odena, C. Olsson, D. Andersen, and I. Goodfellow, "TensorFuzz: Debugging neural networks with coverage-guided fuzzing," in *International Conference on Machine Learning*. PMLR, 2019, pp. 4901–4911.

[12] Y. Tian, K. Pei, S. Jana, and B. Ray, "DeepTest: Automated testing of deep-neural-network-driven autonomous cars," in *Proceedings of the 40th international conference on software engineering*, 2018, pp. 303–314.

[13] Y. Yuan, Q. Pang, and S. Wang, "Unveiling hidden dnn defects with decision-based metamorphic testing," in *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*, 2022, pp. 1–13.

[14] Y. Yuan, S. Wang, M. Jiang, and T. Y. Chen, "Perception matters: Detecting perception failures of vqa models using metamorphic testing," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 16 908–16 917.

[15] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu *et al.*, "DeepGauge: Multi-granularity testing criteria for deep learning systems," in *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*, 2018, pp. 120–131.

[16] H. Guo, C. Tao, Z. Huang, and W. Zou, "Coverage-guided testing for deep learning models: A comprehensive survey," *arXiv preprint arXiv:2507.00496*, 2025.

[17] W. Luo, D. Chai, X. Ruan, J. Wang, C. Fang, and Z. Chen, "Graph-based fuzz testing for deep learning inference engines," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 288–299.

[18] J. Lin, Q. Zhang, J. Li, C. Sun, H. Zhou, C. Luo, and C. Qian, "Automatic library fuzzing through api relation evolvement," in *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS 2025)*, 2025.

[19] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[20] J. Guo, Y. Jiang, Y. Zhao, Q. Chen, and J. Sun, "DLFuzz: Differential fuzzing testing of deep learning systems," in *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2018, pp. 739–743.

[21] X. Xie, L. Ma, F. Juefei-Xu, M. Xue, H. Chen, Y. Liu, J. Zhao, B. Li, J. Yin, and S. See, "DeepHunter: a coverage-guided fuzz testing framework for deep neural networks," in *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*, 2019, pp. 146–157.

[22] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2025, pp. 23–42.

[23] J. Yu, X. Lin, Z. Yu, and X. Xing, "{LLM-Fuzzer}: Scaling assessment of large language model jailbreaks," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 4657–4674.

[24] S. Zhou, T. Li, K. Wang, Y. Huang, L. Shi, Y. Liu, and H. Wang, "Understanding the effectiveness of coverage criteria for large language models: A special angle from jailbreak attacks," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 745–745.

[25] M. H. Asyrofi, Z. Yang, and D. Lo, "CrossASR++: A modular differential testing framework for automatic speech recognition," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 1575–1579.

[26] X. Liu, N. Xu, M. Chen, and C. Xiao, "Autodan: Generating stealthy jailbreak prompts on aligned large language models," in *The Twelfth International Conference on Learning Representations*.

[27] J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine learning testing: Survey, landscapes and horizons," *IEEE Transactions on Software Engineering*, vol. 48, no. 1, pp. 1–36, 2020.

[28] F. Wang, K. Ben, and X. Zhang, "Sr-mt: a metamorphic method to test the robustness of speech recognition software," in *Proceedings of the 7th International Workshop on Metamorphic Testing*, 2022, pp. 15–22.

[29] Y. Tian, S. Ma, M. Wen, Y. Liu, S.-C. Cheung, and X. Zhang, "To what extent do dnn-based image classification models make unreliable inferences?" *Empirical Software Engineering*, vol. 26, no. 5, p. 84, 2021.

[30] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 39–57.

[31] N. Carlini, P. Mishra, T. Vaidya, Y. Zhang, M. Sherr, C. Shields, D. Wagner, and W. Zhou, "Hidden voice commands," in *25th USENIX security symposium (USENIX security 16)*, 2016, pp. 513–530.

[32] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2574–2582.

[33] Z. Zhang, E. Yang, and S. Fang, "Commandergabble: A universal attack against asr systems leveraging fast speech," in *Proceedings of the 37th Annual Computer Security Applications Conference*, 2021, pp. 720–731.

[34] Y. Qin, N. Carlini, G. Cottrell, I. Goodfellow, and C. Raffel, "Imperceptible, robust, and targeted adversarial examples for automatic speech recognition," in *International conference on machine learning*. PMLR, 2019, pp. 5231–5240.

[35] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[36] S. Dola, M. B. Dwyer, and M. L. Soffa, "Input distribution coverage: Measuring feature interaction adequacy in neural network testing," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 3, pp. 1–48, 2023.

[37] M. Usman, Y. Sun, D. Gopinath, R. Dange, L. Manolache, and C. S. Păsăreanu, "An overview of structural coverage metrics for testing neural networks," *International Journal on Software Tools for Technology Transfer*, vol. 25, no. 3, pp. 393–405, 2023.

[38] Z. Ji, P. Ma, Y. Yuan, and S. Wang, "Cc: Causality-aware coverage criterion for deep neural networks," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1788–1800.

[39] Y. Yuan, Q. Pang, and S. Wang, "Revisiting neuron coverage for dnn testing: A layer-wise and distribution-aware criterion," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1200–1212.

[40] M. H. Asyrofi, F. Thung, D. Lo, and L. Jiang, "CrossASR: Efficient differential testing of automatic speech recognition via text-to-speech," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 640–650.

[41] H. You, Z. Wang, J. Chen, S. Liu, and S. Li, "Regression fuzzing for deep learning systems," in *2023 IEEE/ACM 45th International*

Conference on Software Engineering (ICSE). IEEE, 2023, pp. 82–94.

[42] X. Gong, M. Li, Y. Zhang, F. Ran, C. Chen, Y. Chen, Q. Wang, and K. Lam, "PAPILLON: Efficient and stealthy fuzz testing-powered jailbreaks for llms," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security '25)*. Seattle, WA, USA: USENIX Association, Aug. 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/technical-sessions

[43] L. Wang, X. Xie, X. Du, M. Tian, Q. Guo, Z. Yang, and C. Shen, "DistXplore: Distribution-guided testing for evaluating and enhancing deep learning systems," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 68–80.

[44] X. Chen, X. Gao, J. Zhao, K. Ye, and C.-Z. Xu, "Advdiffuser: Natural adversarial example synthesis with diffusion models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4562–4572.

[45] V. Riccio, G. Jahangirova, A. Stocco, N. Humbatova, M. Weiss, and P. Tonella, "Testing machine learning based systems: a systematic mapping," *Empirical Software Engineering*, vol. 25, no. 6, pp. 5193–5254, 2020.

[46] H. B. Braiek and F. Khomh, "On testing machine learning programs," *Journal of Systems and Software*, vol. 164, p. 110542, 2020.

[47] X. Huang, D. Kroening, W. Ruan, J. Sharp, Y. Sun, E. Thamo, M. Wu, and X. Yi, "A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability," *Computer Science Review*, vol. 37, p. 100270, 2020.

[48] J. Wang, H. Qiu, Y. Rong, H. Ye, Q. Li, Z. Li, and C. Zhang, "BET: black-box efficient testing for convolutional neural networks," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 164–175.

[49] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, "Adversarial examples are not bugs, they are features," *Advances in neural information processing systems*, vol. 32, 2019.

[50] Y. Chun Tie, M. Birks, and K. Francis, "Grounded theory research: A design framework for novice researchers," *SAGE open medicine*, vol. 7, p. 2050312118822927, 2019.

[51] C. Xie, J. Wang, Z. Zhang, Z. Ren, and A. Yuille, "Mitigating adversarial effects through randomization," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=Sk9yuql0Z

[52] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *2016 IEEE symposium on security and privacy (SP)*. IEEE, 2016, pp. 582–597.

[53] W. Xu, D. Evans, and Y. Qi, "Feature squeezing: Detecting adversarial examples in deep neural networks," *arXiv preprint arXiv:1704.01155*, 2017.

[54] T. Bai, J. Luo, J. Zhao, B. Wen, and Q. Wang, "Recent advances in adversarial training for adversarial robustness," *arXiv preprint arXiv:2102.01356*, 2021.

[55] B. Wu, S. Wei, M. Zhu, M. Zheng, Z. Zhu, M. Zhang, H. Chen, D. Yuan, L. Liu, and Q. Liu, "Defenses in adversarial machine learning: A survey," *arXiv preprint arXiv:2312.08890*, 2023.

[56] T. Shen, R. Jin, Y. Huang, C. Liu, W. Dong, Z. Guo, X. Wu, Y. Liu, and D. Xiong, "Large language model alignment: A survey," *arXiv preprint arXiv:2309.15025*, 2023.

[57] N. Das, M. Shanbhogue, S.-T. Chen, F. Hohman, S. Li, L. Chen, M. E. Kounavis, and D. H. Chau, "Shield: Fast, practical defense and vaccination for deep learning using jpeg compression," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 196–204.

[58] N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. Goodfellow, A. Madry, and A. Kurakin, "On evaluating adversarial robustness," *arXiv preprint arXiv:1902.06705*, 2019.

[59] L. Sun, M. Tan, and Z. Zhou, "A survey of practical adversarial example attacks," *Cybersecurity*, vol. 1, no. 1, p. 9, 2018.

[60] Y. Chen, X. Yuan, J. Zhang, Y. Zhao, S. Zhang, K. Chen, and X. Wang, "{Devil's} whisper: A general approach for physical adversarial attacks against commercial black-box speech recognition devices," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 2667–2684.

[61] Z. Yu, Y. Chang, N. Zhang, and C. Xiao, "{SMACK}: Semantically meaningful adversarial audio attack," in *32nd USENIX security symposium (USENIX security 23)*, 2023, pp. 3799–3816.

[62] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, and D. Song, "Robust physical-world attacks on deep learning visual classification," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 1625–1634.

[63] X. Du, X. Xie, Y. Li, L. Ma, J. Zhao, and Y. Liu, "DeepCruiser: Automated guided testing for stateful deep learning systems," *arXiv preprint arXiv:1812.05339*, 2018.

[64] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *CVPR 2011*. IEEE, 2011, pp. 1521–1528.

[65] N. Papernot, P. McDaniel, and I. Goodfellow, "Transferability in machine learning: from phenomena to black-box attacks using adversarial samples," *arXiv preprint arXiv:1605.07277*, 2016.

[66] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=Sys6GJqxl

[67] W. Liang, Y. Zhang, H. Cao, B. Wang, D. Y. Ding, X. Yang, K. Vodrahalli, S. He, D. S. Smith, Y. Yin *et al.*, "Can large language models provide useful feedback on research papers? a large-scale empirical analysis," *NEJM AI*, vol. 1, no. 8, p. AIoa2400196, 2024.

[68] N. Thakkar, M. Yuksekgonul, J. Silberg, J. Zou, C. Vondrick, R. Yu, V. Peng, F. Sha, and A. Garg, "Leveraging llm feedback to enhance review quality," 2025, iCLR Blog, April 15 2025. [Online]. Available: https://blog.iclr.cc/2025/04/15/leveraging-llm-feedback-to-enhance-review-quality/

[69] J. Lála, O. O'Donoghue, A. Shtedritski, S. Cox, S. G. Rodriques, and A. D. White, "Paperqa: Retrieval-augmented generative agent for scientific research," *arXiv preprint arXiv:2312.07559*, 2023.

[70] B. Bohnet, V. Q. Tran, P. Verga, R. Aharoni, D. Andor, L. B. Soares, M. Ciaramita, J. Eisenstein, K. Ganchev, J. Herzig *et al.*, "Attributed question answering: Evaluation and modeling for attributed large language models," *arXiv preprint arXiv:2212.08037*, 2022.

[71] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[72] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, "OpenAI GPT-5 System Card," *arXiv preprint arXiv:2601.03267*, 2025.

[73] OpenAI, "OpenAI o3 and o4-mini System Card," OpenAI, System Card, Apr. 2025. [Online]. Available: https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf

[74] Computing Research and Education Association of Australasia (CORE), "Core conference rankings portal," https://portal.core.edu.au/conf-ranks/, 2025, accessed: 2025-05-26.

[75] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *arXiv preprint arXiv:2002.12543*, 2020.

[76] X. Du, X. Xie, Y. Li, L. Ma, Y. Liu, and J. Zhao, "DeepStellar: Model-based quantitative analysis of stateful deep learning systems," in *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2019, pp. 477–487.

[77] T. Woodlief, S. Elbaum, and K. Sullivan, "Semantic image fuzzing of ai perception systems," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1958–1969.

[78] M. H. Asyrofi, Z. Yang, J. Shi, C. W. Quan, and D. Lo, "Can differential testing improve automatic speech recognition systems?" in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2021, pp. 674–678.

[79] Z. Li, M. Pan, T. Zhang, and X. Li, "Testing DNN-based autonomous driving systems under critical environmental conditions," in *International conference on machine learning*. PMLR, 2021, pp. 6471–6482.

[80] X. Xie, T. Li, J. Wang, L. Ma, Q. Guo, F. Juefei-Xu, and Y. Liu, "Npc: Neuron path coverage via characterizing decision logic of deep neural networks," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 3, pp. 1–27, 2022.

[81] J. Kim, R. Feldt, and S. Yoo, "Guiding deep learning system testing using surprise adequacy," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 1039–1049.
[82] X. Zhang, X. Xie, L. Ma, X. Du, Q. Hu, Y. Liu, J. Zhao, and M. Sun, "Towards characterizing adversarial defects of deep learning software from the lens of uncertainty," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 739–751.
[83] Q. Pang, Y. Yuan, and S. Wang, "Mdpfuzz: testing models solving markov decision processes," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 378–390.
[84] W. Huang, Y. Sun, X. Zhao, J. Sharp, W. Ruan, J. Meng, and X. Huang, "Coverage-guided testing for recurrent neural networks," *IEEE Transactions on Reliability*, vol. 71, no. 3, pp. 1191–1206, 2021.
[85] Y. Zhang, L. Xu, A. Mendoza, G. Yang, P. Chinprutthiwong, and G. Gu, "Life after speech recognition: Fuzzing semantic misinterpretation for voice assistant applications," in *Proc. of the Network and Distributed System Security Symposium (NDSS'19)*, 2019.
[86] Z. Liu, Y. Feng, Y. Yin, J. Sun, Z. Chen, and B. Xu, "QATest: A uniform fuzzing framework for question answering systems," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
[87] E. Dai, W. Mo, K. Hu, X. Zhu, X. Xiao, S. Wen, S. Wang, and Y. Xiang, "TWINFUZZ: Dual-model fuzzing for robustness generalization in deep learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 44, 2026, pp. 37 314–37 322.
[88] S. Lee, S. Cha, D. Lee, and H. Oh, "Effective white-box testing of deep neural networks with adaptive neuron-selection strategy," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 165–176.
[89] H. Zhang, D. Wang, Y. Liu, K. Chen, J. Wang, X. Ying, L. Liu, and W. Wang, "ORFuzz: Fuzzing the" other side" of llm safety-testing over-refusal," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 1869–1880.
[90] Y. Yang, B. Hui, H. Yuan, N. Gong, and Y. Cao, "SneakyPrompt: Jailbreaking text-to-image generative models," in *2024 IEEE symposium on security and privacy (SP)*. IEEE, 2024, pp. 897–912.
[91] J. He, Z. Yang, J. Shi, C. Yang, K. Kim, B. Xu, X. Zhou, and D. Lo, "Curiosity-driven testing for sequential decision-making process," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–14.
[92] D. H. X. Yuen, A. Y. C. Pang, Z. Yang, C. Y. Chong, M. K. Lim, and D. Lo, "Asdf: A differential testing framework for automatic speech recognition systems," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2023, pp. 461–463.
[93] P. Zhang, B. Ren, H. Dong, and Q. Dai, "CAGFuzz: coverage-guided adversarial generative fuzzing testing for image-based deep learning systems," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4630–4646, 2021.
[94] X. Gao, R. K. Saha, M. R. Prasad, and A. Roychoudhury, "Fuzz testing based data augmentation to improve robustness of deep neural networks," ser. ICSE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1147–1158. [Online]. Available: https://doi.org/10.1145/3377811.3380415
[95] Y. Yuan, S. Wang, and Z. Su, "See the forest, not trees: Unveiling and escaping the pitfalls of error-triggering inputs in neural network testing," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1605–1617.
[96] D. Ma, J. Guo, Y. Jiang, and X. Jiao, "HDTest: Differential fuzz testing of brain-inspired hyperdimensional computing," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 391–396.
[97] H. B. Braiek and F. Khomh, "DeepEvolution: A search-based testing approach for deep neural networks," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019, pp. 454–458.
[98] J. Wang, J. Chen, Y. Sun, X. Ma, D. Wang, J. Sun, and P. Cheng, "RobOT: Robustness-oriented testing for deep learning systems," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 300–311.
[99] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.
[100] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
[101] A. W. Rix, J. G. Beerends, M. P. Hollier, and A. P. Hekstra, "Perceptual evaluation of speech quality (pesq)-a new method for speech quality assessment of telephone networks and codecs," in *2001 IEEE international conference on acoustics, speech, and signal processing. Proceedings (Cat. No. 01CH37221)*, vol. 2. IEEE, 2001, pp. 749–752.
[102] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, "An algorithm for intelligibility prediction of time–frequency weighted noisy speech," *IEEE Transactions on audio, speech, and language processing*, vol. 19, no. 7, pp. 2125–2136, 2011.
[103] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling," *Computer Speech & Language*, vol. 13, no. 4, pp. 359–394, 1999.
[104] X. Yuan, P. He, Q. Zhu, and X. Li, "Adversarial examples: Attacks and defenses for deep learning," *IEEE transactions on neural networks and learning systems*, vol. 30, no. 9, pp. 2805–2824, 2019.

APPENDIX

B. LLM VS. EXPERT ASSESSMENT ANALYSIS

Tables VII and VIII summarize agreement between LLM-based preliminary assessments and expert-corrected final assessments across papers and metrics, respectively. Overall agreement was high (87%), with remaining discrepancies mainly arising from evidence misinterpretation or selection from irrelevant sections.

In several cases, the LLM-assisted pipeline relied on contextual cues or implicit assumptions rather than the formal codebook criteria. For example, in *DeepTest*, it assigned a *medium* rating for targeted attack discovery because the target models belonged to a safety-critical domain (autonomous driving), even though the fuzzer performs untargeted exploration. In *AutoDAN*, it misinterpreted hypotheses about transferability as diagnostic insight into failure root causes. Similarly, it penalized *CureFuzz* for ignoring I/O serialization effects, overlooking that its validity checks keep feature vectors within legal bounds and do not require clipping or rounding.

In other cases, the pipeline extracted evidence from irrelevant sections, leading to incorrect interpretations despite internally consistent reasoning. For instance, in *Zhou et al.*, the LLM-assisted pipeline assigned a low rating on the input plausibility metric by relying on the jailbreak detection section, where prompts contain meaningless adversarial suffixes, instead of the fuzzing method description. In contrast, human experts corrected this by examining the fuzzing design section and assigning a high rating, as the fuzzer synthesizes natural, semantically coherent prompts using an LLM.

Overall, these findings demonstrate that while LLM assistance substantially accelerates metric-relevant evidence localization, expert oversight remains essential for ensuring contextual accuracy and metric-aligned interpretation.

Listing 1. LLM Prompt Templates

```
# Evaluator Prompt
You are an expert in DNN fuzzing research.
You are given:
1. Metric codebook (definition and evaluation guidance
   for each compliance level).
2. The full text of an DNN fuzzing research paper.
Your task is to evaluate the method proposed in the paper
against the specified metric.
Instructions:
```

TABLE VII
PER-PAPER AGREEMENT BETWEEN LLM-GENERATED LABELS AND EXPERT-CORRECTED LABELS ACROSS ALL METRICS. GRAY CELLS INDICATE A DISAGREEMENT. (**L**=LLM-BASED PRELIMINARY ASSESSMENT, **H**=HUMAN EXPERT FINAL ASSESSMENT)

Paper	FS		TA		IP		FR		FD		AT	
	L	H	L	H	L	H	L	H	L	H	L	H
PAIR [22]	●	●	●	◑	●	●	●	●	◑	○	◑	◑
PAPILLON [42]	●	●	●	●	●	●	●	●	◑	◑	◑	◑
ORFuzz [89]	○	○	◑	◑	◑	●	●	●	◑	◑	◑	◑
AutoDAN [26]	●	●	●	●	●	●	●	●	◑	○	◑	◑
SneakyPrompt [90]	●	●	●	●	●	●	●	●	◑	◑	○	○
LLM-Fuzzer [23]	●	●	●	◑	●	●	●	●	○	○	●	●
CureFuzz [91]	◑	◑	◑	◑	◑	●	●	●	◑	◑	○	○
DRFuzz [41]	○	○	◑	○	◑	●	○	○	◑	●	○	○
ASDF [92]	○	○	○	○	●	●	●	●	◑	◑	○	○
MDPFuzz [83]	◑	◑	◑	◑	●	●	●	●	◑	◑	○	○
BET [48]	○	○	●	●	●	●	●	●	○	○	○	○
QATest [86]	○	○	○	○	◑	◑	●	●	◑	◑	○	○
semSensFuzz [77]	○	○	○	○	●	●	◑	◑	◑	◑	○	○
CAGFuzz [93]	○	○	○	○	◑	◑	○	○	○	○	●	○
CrossASR++ [25]	○	○	○	○	●	●	●	●	○	○	○	○
Asyrofi et al. [78]	○	○	○	○	●	●	●	●	○	○	○	○
Sensei [94]	○	○	○	○	●	●	◑	◑	○	○	○	○
KuK [82]	●	●	○	○	●	●	○	◑	◑	◑	◑	◑
CrossASR [40]	○	○	○	○	●	●	●	●	○	○	○	○
LipFuzzer [85]	◑	◑	○	○	◑	●	●	●	◑	◑	○	○
Zhou et al. [24]	●	●	◑	◑	◑	◑	●	●	◑	◑	○	○
Yuan et al. [95]	○	○	○	○	◑	◑	○	○	●	◑	○	○
NLC [39]	○	○	○	○	◑	◑	◑	◑	◑	◑	○	○
DistXplore [43]	○	○	●	●	◑	◑	◑	◑	●	●	○	○
TESTRNN [84]	◑	○	○	○	◑	●	●	◑	◑	◑	○	○
TACTIC [79]	○	○	◑	○	●	●	○	○	◑	◑	○	○
HDTest [96]	○	○	○	○	●	●	◑	◑	◑	◑	○	○
DeepHunter [21]	○	○	○	○	●	●	◑	◑	◑	◑	◑	○
TensorFuzz [11]	○	○	●	◑	●	●	◑	◑	◑	◑	○	○
DeepStellar [76]	○	○	○	○	◑	◑	◑	◑	◑	◑	○	○
DeepEvolution [97]	○	○	○	○	●	●	◑	◑	○	○	○	○
DeepTest [12]	◑	◑	○	○	◑	◑	◑	◑	◑	◑	○	○
DeepCruiser [63]	○	○	○	○	●	●	◑	◑	◑	◑	○	○
TWINFUZZ [87]	●	●	○	○	●	●	○	○	○	○	○	○
RobOT [98]	○	○	○	○	◑	◑	○	○	◑	◑	○	○
ADAPT [88]	○	○	○	○	●	●	○	○	◑	◑	○	○
DLFuzz [20]	○	○	○	○	●	●	○	○	○	○	○	○
DeepXplore [10]	◑	○	○	○	◑	◑	●	●	●	●	○	○

●: High ◑: Medium ○: Low

TABLE VIII
LLM VS HUMAN AGREEMENT ON EVALUATION METRICS. EACH CELL REPORTS THE AGREEMENT AS COUNT / TOTAL (PERCENTAGE%) ACROSS 32 DNN FUZZING STUDIES.

Metric	Agreement
Failure Severity	36/38 (94.7%)
Targeted Attack Discovery	33/38 (86.8%)
Input Plausibility	33/38 (86.8%)
Failure Reproducibility	36/38 (94.7%)
Failure Diagnostics	34/38 (89.5%)
Attack Transferability	36/38 (94.7%)
Overall	**208/228 (91.2%)**

```
- Base your reasoning strictly on the paper content.
- Focus on technical and methodological sections.
- Assign one of the allowed values from the codebook.
- Justify your choice with explanation.
- Support it with direct evidence from paper.
Output Format:
- Return a JSON object using this schema:
{
  "value": "<value>", "why": "<Explanation>",
  "evidence": "<page, section, supporting quote>"
}

# Arbitrator Prompt
You are expert resolving disagreement between assessments
You are given:
1. Metric codebook (definition and evaluation guidance
   for each compliance level).
2. The full text of a DNN fuzzing research paper.
3. Two assessments with conflicting verdicts.
Instructions:
- Review both assessments using the codebook and paper.
- Assign a value and provide rationale.
- Justify your conclusion with explanation.
- Support it with direct evidence from the paper.
Output Format:
- Return a JSON object using this schema:
{
  "value": "<value>", "why": "<Explanation>",
  "evidence": "<page, section, supporting quote>"
}
```

Listing 2. LLM Assessment Outputs

```
# Evaluator A (ChatGPT:gpt-5-mini)
{
 "value": "Low",
 "why": "The paper studies coverage criteria and
   jailbreak detection for LLMs using [truncated]",
 "evidence": "Page 731, Section II-A (Model Inference
   Process): \"The text is tokenized into discrete
   elements, each mapped to a dense [truncated]"
}

# Evaluator B (Gemini:gemini-2.5-pro)
{
 "value": "High",
 "why": "The paper's methodology for generating and
   selecting test cases for Large Language Models (LLMs)
   operates entirely within the text domain [truncated]",
 "evidence": "Page 4, Section III-A, 'Findings': 'Attack
   queries are generated by appending adversarial
   suffixes to rejected queries using [truncated]"
}

# Arbitrator Output (ChatGPT:o4-mini)
{
  "value": "High",
  "why": "All of the paper's test generation and
    evaluation is performed in the textual domain: inputs
    are discrete text prompts that are [truncated]",
  "evidence": "Page 731, Section III-A: "Attack queries
    are generated by appending adversarial suffixes to
    rejected queries using GCG [7]. [truncated]"
}
```

M.	L.	Paper	Evidence
Failure Severity	●	PAIR [22]	Abstract and Sec. IV describe semantic jailbreaks against safety-aligned LLMs; Table III reports jailbreak success on GPT-3.5, GPT-4, Claude, and Gemini; Sec. IV-C evaluates PAIR against SmoothLLM and perplexity-filter defenses, showing failures that bypass safeguards on defended models.
	◑	CureFuzz [91]	Sec. 3.1 defines the goal as uncovering crash-triggering scenarios in sequential decision-making systems; Sec. 4.2 instantiates these failures as vehicle collisions, aircraft collisions, and robot falls. The paper targets catastrophic unsafe behavior, but not safeguard bypass on defended models.
	○	DeepHunter [21]	Sec. 3.2 defines erroneous behavior as a mutant that changes the model prediction under metamorphic mutation, and Sec. 5.4 highlights detection of quantization-related errors. The evaluation focuses on robustness-related mispredictions rather than unsafe behavior or safeguard bypass.
Targeted Attack Discovery	●	AutoDAN [26]	Sec. 3.1 specifies a target response prefix such as "Sure, here is how to [Qi]..."; Sec. 3.3 optimizes the conditional probability of that target sequence; Alg. 2 and Sec. 3.5 steer the search away from refusal outputs. The search is explicitly guided toward a predefined attacker-chosen output.
	◑	PAIR [22]	Sec. II defines each jailbreak by an objective O describing the harmful content to elicit, and Sec. IV evaluates prompts against behaviors from JailbreakBench. PAIR steers toward a class of unsafe outcomes tied to the behavior objective, rather than a single fixed output string.
	○	DeepXplore [10]	Sec. 3 and Sec. 4.2 maximize differential behaviors and neuron coverage so that one DNN predicts differently from others. The optimization seeks any disagreement-inducing input, not a predefined attacker-chosen output or unsafe outcome class.
Input Plausibility	●	AutoDAN [26]	Sec. 3.2 uses LLM-based revision to preserve the logical flow and meaning of handcrafted jailbreak prompts; Sec. 3.4 reuses LLM-based diversification as the mutation operator; Table 1 and Sec. 4.2 report lower perplexity and semantically meaningful prompts. The method explicitly preserves semantic plausibility during prompt generation.
	◑	NLC [39]	Sec. VI-B applies realistic image transformations and accepts a mutation only if `is_valid()` holds, using bounds on changed pixels and pixel magnitude; the same section reports naturalness via IS/FID. These are per-mutation plausibility checks, but the paper does not enforce plausibility relative to the original seed across the full fuzzing process.
	○	None	No evaluated paper received a **low** (○) label on this metric, indicating that all assessed DNN fuzzers include at least some mechanism intended to preserve input plausibility.
Failure Reproducibility	●	CrossASR++ [25]	Sec. 2.1 generates test cases directly as TTS audio files (`generateAudio(texts)`), and Sec. 2.4.2 states that the repository provides all generated audios for reuse. Because the fault-inducing inputs are produced and reused in serialized audio form, the discovered failures are reproducible under standard I/O.
	◑	NLC [39]	Sec. VI-B uses metamorphic image transformations such as contrast, brightness, translation, rotation, blurring, and stylizing, and bounds pixel changes with `is_valid()`. However, the paper does not discuss saving/reloading inputs or handling serialization effects such as rounding or quantization.
	○	DLFuzz [20]	Sec. 3.2 mutates inputs by adding gradient-derived perturbations in memory and keeps the l_2 distance small; Sec. 4 reports adversarial inputs with tiny perturbations. The paper never discusses clipping, rounding, quantization, or re-evaluating saved inputs, so reproducibility under standard I/O is not addressed.
Failure Diagnostics	●	DRFuzz [41]	Sec. IV-B1 analyzes which classes contain the most regression faults and links them to overfitting; Sec. V-B uses Grad-CAM to show that different faulty behaviors are associated with different salient features. The paper goes beyond reporting failures to explain why regression faults occur.
	◑	DeepXplore [10]	Sec. 4.1 defines neuron coverage and Sec. 7.1 analyzes how neuron activations differ across classes; Tables 3 and 4 identify concrete feature changes associated with misclassification; Sec. 7.3 connects discovered differences to poisoned training samples. The paper provides diagnostic insight through internal signals and failure-pattern analysis, but not a full causal account of all failure modes.
	○	PAIR [22]	Secs. IV-A to IV-E report jailbreak success rates across models, categories, defenses, and attacker variants, but the paper does not analyze model internals, failure patterns, or underlying vulnerabilities beyond high-level speculation. It reports failures effectively without diagnosing why they occur.
Attack Transferability	●	LLM-Fuzzer [23]	Experiment III explicitly optimizes for templates that succeed across multiple source LLMs by assigning zero reward to templates whose successful jailbreaks come from only one or two models; Sec. 4.3 then evaluates the resulting templates on additional downstream LLMs and reports strong transfer in Fig. 4. Transferability is both measured and built into the search objective.
	◑	PAIR [22]	Sec. IV-B reuses successful jailbreak prompts found on GPT-4 and Vicuna to attack other downstream LLMs, and Table VI reports the transferred jailbreak rates. The paper measures cross-model transfer, but the method itself is not explicitly designed around a transfer-focused objective.
	○	DeepXplore [10]	Sec. 3 and Algorithm 1 define DeepXplore over a fixed set of multiple DNNs used as cross-referencing oracles during search, and Sec. 8 notes that differential testing requires at least two DNNs with the same functionality. The paper evaluates disagreement among the models used in the search, but does not reuse discovered inputs to test transfer to unseen target models.

TABLE IX

WORKED EXAMPLES SHOWING HOW THE FINAL FUZZCHECK.AI CODEBOOK IS APPLIED TO REPRESENTATIVE PAPERS. EACH ENTRY REPORTS THE FINAL MANUAL LABEL TOGETHER WITH THE PAPER EVIDENCE USED TO JUSTIFY IT.